MDN JavaScript Documentation

Volume 9 of 9 — Built-in objects: TypedArray – WeakSet

Unofficial print compilation. This volume contains 77 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

TypedArray 3
TypedArray.from() 13
TypedArray.of() 17
TypedArray.BYTES_PER_ELEMENT 19
TypedArray[Symbol.species] 21
TypedArray.prototype.at() 23
TypedArray.prototype.copyWithin() 25
TypedArray.prototype.entries() 27
TypedArray.prototype.every() 29
TypedArray.prototype.fill() 31
TypedArray.prototype.filter() 33
TypedArray.prototype.find() 35
TypedArray.prototype.findIndex() 37
TypedArray.prototype.findLast() 40
TypedArray.prototype.findLastIndex() 42
TypedArray.prototype.forEach() 45
TypedArray.prototype.includes() 47
TypedArray.prototype.indexOf() 49
TypedArray.prototype.join() 51
TypedArray.prototype.keys() 52
TypedArray.prototype.lastIndexOf() 54
TypedArray.prototype.map() 56
TypedArray.prototype.reduce() 58
TypedArray.prototype.reduceRight() 61
TypedArray.prototype.reverse() 63
TypedArray.prototype.set() 64
TypedArray.prototype.slice() 66
TypedArray.prototype.some() 68
TypedArray.prototype.sort() 70
TypedArray.prototype.subarray() 72
TypedArray.prototype.toLocaleString() 74
TypedArray.prototype.toReversed() 76
TypedArray.prototype.toSorted() 77
TypedArray.prototype.toString() 79
TypedArray.prototype.values() 80
TypedArray.prototype.with() 82
TypedArray.prototype[Symbol.iterator]() 84
TypedArray.prototype.buffer 86
TypedArray.prototype.byteLength 87
TypedArray.prototype.byteOffset 89
TypedArray.prototype.length 90
TypeError 92
TypeError() constructor 94
Uint8Array 96
Uint8Array() constructor 98
Uint8Array.fromBase64() 100
Uint8Array.fromHex() 104
Uint8Array.prototype.setFromBase64() 106
Uint8Array.prototype.setFromHex() 109
Uint8Array.prototype.toBase64() 111
Uint8Array.prototype.toHex() 114
Uint8ClampedArray 116
Uint8ClampedArray() constructor 118
Uint16Array 121
Uint16Array() constructor 123
Uint32Array 125
Uint32Array() constructor 127
undefined 129
unescape() 133
URIError 135
URIError() constructor 137
WeakMap 139
WeakMap() constructor 145
WeakMap.prototype.delete() 147
WeakMap.prototype.get() 149
WeakMap.prototype.getOrInsert() 151
WeakMap.prototype.getOrInsertComputed() 153
WeakMap.prototype.has() 155
WeakMap.prototype.set() 157
WeakRef 159
WeakRef() constructor 162
WeakRef.prototype.deref() 163
WeakSet 166
WeakSet() constructor 169
WeakSet.prototype.add() 170
WeakSet.prototype.delete() 172
WeakSet.prototype.has() 173

JavaScript › Reference › Standard built-in objects

TypedArray

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

A ***TypedArray*** object describes an array-like view of an underlying binary data buffer. There is no global property named `TypedArray`, nor is there a directly visible `TypedArray` constructor. Instead, there are a number of different global properties, whose values are typed array constructors for specific element types, listed below. On the following pages you will find common properties and methods that can be used with any typed array containing elements of any type.

Description

The `TypedArray` constructor (often referred to as `%TypedArray%` to indicate its "intrinsicness", since it does not correspond to any global exposed to a JavaScript program) serves as the common superclass of all `TypedArray` subclasses. Think about `%TypedArray%` as an "abstract class" providing a common interface of utility methods for all typed array subclasses. This constructor is not directly exposed: there is no global `TypedArray` property. It is only accessible through `Object.getPrototypeOf(Int8Array)` and similar.

When creating an instance of a `TypedArray` subclass (e.g., `Int8Array`), an array buffer is created internally in memory or, if an `ArrayBuffer` object is given as constructor argument, that `ArrayBuffer` is used instead. The buffer address is saved as an internal property of the instance and all the methods of `%TypedArray%.prototype` will set and get values based on that array buffer address.

TypedArray objects

Type	Value Range	Size in bytes	Web IDL type
`Int8Array`	-128 to 127	1	`byte`
`Uint8Array`	0 to 255	1	`octet`
`Uint8ClampedArray`	0 to 255	1	`octet`
`Int16Array`	-32768 to 32767	2	`short`
`Uint16Array`	0 to 65535	2	`unsigned short`
`Int32Array`	-2147483648 to 2147483647	4	`long`
`Uint32Array`	0 to 4294967295	4	`unsigned long`
`Float16Array`	`-65504` to `65504`	2	N/A
`Float32Array`	`-3.4e38` to `3.4e38`	4	`unrestricted float`
`Float64Array`	`-1.8e308` to `1.8e308`	8	`unrestricted double`
`BigInt64Array`	-2^{63} to $2^{63} - 1$	8	`bigint`
`BigUint64Array`	0 to $2^{64} - 1$	8	`bigint`

Value encoding and normalization

All typed arrays operate on `ArrayBuffers`, where you can observe the exact byte representation of each element, so how the numbers are encoded in binary format is significant.

- Unsigned integer arrays (`Uint8Array`, `Uint16Array`, `Uint32Array`, and `BigUint64Array`) store the number directly in binary.

- Signed integer arrays (`Int8Array`, `Int16Array`, `Int32Array`, and `BigInt64Array`) store the number using two's complement.
- Floating-point arrays (`Float16Array`, `Float32Array`, and `Float64Array`) store the number using IEEE 754 floating-point format. The `Number` reference has more information about the exact format. JavaScript numbers use double precision floating point format by default, which is the same as `Float64Array`. `Float32Array` uses 23 (instead of 52) bits for the mantissa and 8 (instead of 11) bits for the exponent. `Float16Array` uses 10 bits for the mantissa and 5 bits for the exponent. Note that the spec requires all `NaN` values to use the same bit encoding, but the exact bit pattern is implementation-dependent.
- `Uint8ClampedArray` is a special case. It stores the number in binary like `Uint8Array` does, but when you store a number outside the range, it *clamps* the number to the range 0 to 255 by mathematical value, instead of truncating the most significant bits.

All typed arrays except `Int8Array`, `Uint8Array`, and `Uint8ClampedArray` store each element using multiple bytes. These bytes can either be ordered from most significant to least significant (big-endian) or from least significant to most significant (little-endian). See Endianness for more explanation. Typed arrays always use the platform's native byte order. If you want to specify the endianness when writing and reading from buffers, you should use a `DataView` instead.

When writing to these typed arrays, values that are outside the representable range are normalized.

- All integer arrays (except `Uint8ClampedArray`) use fixed-width number conversion, which first truncates the decimal part of the number and then takes the lowest bits.
- `Uint8ClampedArray` first clamps the number to the range 0 to 255 (values greater than 255 become 255 and values less than 0 become 0). It then *rounds* (instead of flooring) the result to the nearest integer, with half-to-even; meaning if the number is exactly between two integers, it rounds to the nearest even integer. For example, `0.5` becomes `0`, `1.5` becomes `2`, and `2.5` becomes `2`.
- `Float16Array` and `Float32Array` perform a "round to even" to convert 64-bit floating point numbers to 32-bit and 16-bit. This is the same algorithm as provided by `Math.fround()` and `Math.f16round()`.

Behavior when viewing a resizable buffer

When a `TypedArray` is created as a view of a resizable buffer, resizing the underlying buffer will have different effects on the size of the `TypedArray` depending on whether the `TypedArray` is constructed as length-tracking.

If a typed array is created without a specific size by omitting the third parameter or passing `undefined`, the typed array will become *length-tracking*, and will automatically resize to fit the underlying `buffer` as the latter is resized:

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2

buffer.resize(12);

console.log(float32.byteLength); // 12
console.log(float32.length); // 3
```

If a typed array is created with a specific size using the third `length` parameter, it won't resize to contain the `buffer` as the latter is grown:

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer, 0, 2);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2
console.log(float32[0]); // 0, the initial value

buffer.resize(12);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2
console.log(float32[0]); // 0, the initial value
```

When a `buffer` is shrunk, the viewing typed array may become out of bounds, in which case the typed array's observed size will decrease to 0. This is the only case where a non-length-tracking typed array's length may change.

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer, 0, 2);

buffer.resize(7);

console.log(float32.byteLength); // 0
console.log(float32.length); // 0
console.log(float32[0]); // undefined
```

If you then grow the `buffer` again to bring the typed array back in bounds, the typed array's size will be restored to its original value.

```js
buffer.resize(8);

console.log(float32.byteLength); // 8
console.log(float32.length); // 2
console.log(float32[0]); // 0 - back in bounds again!
```

The same can happen for length-tracking typed arrays as well, if the buffer is shrunk beyond the `byteOffset`.

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const float32 = new Float32Array(buffer, 4);
// float32 is length-tracking, but it only extends from the 4th byte
// to the end of the buffer, so if the buffer is resized to be shorter
// than 4 bytes, the typed array will become out of bounds
buffer.resize(3);
console.log(float32.byteLength); // 0
```

Constructor

This object cannot be instantiated directly — attempting to construct it with `new` throws a `TypeError`.

```js
new (Object.getPrototypeOf(Int8Array))();
// TypeError: Abstract class TypedArray not directly constructable
```

Instead, you create an instance of a typed array of a particular type, such as an `Int8Array` or a `BigInt64Array`. These objects all have a common syntax for their constructors:

```js
new TypedArray()
new TypedArray(length)
new TypedArray(typedArray)
new TypedArray(object)

new TypedArray(buffer)
new TypedArray(buffer, byteOffset)
new TypedArray(buffer, byteOffset, length)
```

Where `TypedArray` is a constructor for one of the concrete types.

> **Note:** All `TypedArray` subclasses' constructors can only be constructed with `new`. Attempting to call one without `new` throws a `TypeError`.

Parameters

`typedArray`

When called with an instance of a `TypedArray` subclass, the `typedArray` gets copied into a new typed array. For a non-bigint `TypedArray` constructor, the `typedArray` parameter can only be of one of the non-bigint types (such as `Int32Array`). Similarly, for a bigint `TypedArray` constructor (`BigInt64Array` or `BigUint64Array`), the `typedArray` parameter can only be of one of the bigint types. Each value in `typedArray` is converted to the corresponding type of the constructor before being copied into the new array. The length of the new typed array will be same as the length of the `typedArray` argument.

`object`

When called with an object that's not a `TypedArray` instance, a new typed array is created in the same way as the `TypedArray.from()` method.

`length` Optional

When called with a non-object, the parameter will be treated as a number specifying the length of the typed array. An internal array buffer is created in memory, of size `length` multiplied by `BYTES_PER_ELEMENT` bytes, filled with zeros. Omitting all parameters is equivalent to using `0` as `length`.

`buffer`, `byteOffset` Optional**, `length`** Optional

When called with an `ArrayBuffer` or `SharedArrayBuffer` instance, and optionally a `byteOffset` and a `length` argument, a new typed array view is created that views the specified buffer. The `byteOffset` (in bytes) and `length` (in number of elements, each occupying `BYTES_PER_ELEMENT` bytes) parameters specify the memory range that will be exposed by the typed array view. If both

are omitted, all of `buffer` is viewed; if only `length` is omitted, the remainder of `buffer` starting from `byteOffset` is viewed. If `length` is omitted, the typed array becomes length-tracking.

Exceptions

All `TypeArray` subclass constructors operate in the same way. They would all throw the following exceptions:

`TypeError`

Thrown in one of the following cases:

- A `typedArray` is passed but it is a bigint type while the current constructor is not, or vice versa.
- A `typedArray` is passed but the buffer it's viewing is detached, or a detached `buffer` is directly passed.

`RangeError`

Thrown in one of the following cases:

- The new typed array's length is too large.
- The length of `buffer` (if the `length` parameter is not specified) or `byteOffset` is not an integral multiple of the new typed array's element size.
- `byteOffset` is not a valid array index (an integer between 0 and 2^{53} - 1).
- When creating a view from a buffer, the bounds are outside the buffer. In other words, `byteOffset + length * TypedArray.BYTES_PER_ELEMENT > buffer.byteLength`.

Static properties

These properties are defined on the `TypedArray` constructor object and are thus shared by all `TypedArray` subclass constructors.

`TypedArray[Symbol.species]`

The constructor function used to create derived objects.

All `TypedArray` subclasses also have the following static properties:

`TypedArray.BYTES_PER_ELEMENT`

Returns a number value of the element size for the different `TypedArray` objects.

Static methods

These methods are defined on the `TypedArray` constructor object and are thus shared by all `TypedArray` subclass constructors.

`TypedArray.from()`

Creates a new `TypedArray` from an array-like or iterable object. See also `Array.from()`.

`TypedArray.of()`

Creates a new `TypedArray` with a variable number of arguments. See also `Array.of()`.

Instance properties

These properties are defined on `TypedArray.prototype` and shared by all `TypedArray` subclass instances.

`TypedArray.prototype.buffer`

Returns the `ArrayBuffer` referenced by the typed array.

`TypedArray.prototype.byteLength`

Returns the length (in bytes) of the typed array.

`TypedArray.prototype.byteOffset`

Returns the offset (in bytes) of the typed array from the start of its `ArrayBuffer`.

`TypedArray.prototype.constructor`

The constructor function that created the instance object. `TypedArray.prototype.constructor` is the hidden `TypedArray` constructor function, but each typed array subclass also defines its own `constructor` property.

`TypedArray.prototype.length`

Returns the number of elements held in the typed array.

`TypedArray.prototype[Symbol.toStringTag]`

The initial value of the `TypedArray.prototype[Symbol.toStringTag]` property is a getter that returns the same string as the typed array constructor's name. It returns `undefined` if the `this` value is not one of the typed array subclasses. This property is used in `Object.prototype.toString()`. However, because `TypedArray` also has its own `toString()` method, this property is not used unless you call `Object.prototype.toString.call()` with a typed array as `thisArg`.

All `TypedArray` subclasses also have the following instance properties:

`TypedArray.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size for the different `TypedArray` objects.

Instance methods

These methods are defined on the `TypedArray` prototype object and are thus shared by all `TypedArray` subclass instances.

`TypedArray.prototype.at()`

Takes an integer value and returns the item at that index. This method allows for negative integers, which count back from the last item.

`TypedArray.prototype.copyWithin()`

Copies a sequence of array elements within the array. See also `Array.prototype.copyWithin()`.

`TypedArray.prototype.entries()`

Returns a new *array iterator* object that contains the key/value pairs for each index in the array. See also `Array.prototype.entries()`.

`TypedArray.prototype.every()`

Returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`. See also `Array.prototype.every()`.

`TypedArray.prototype.fill()`

Fills all the elements of an array from a start index to an end index with a static value. See also `Array.prototype.fill()`.

`TypedArray.prototype.filter()`

Creates a new array with all of the elements of this array for which the provided filtering function returns `true`. See also `Array.prototype.filter()`.

`TypedArray.prototype.find()`

Returns the first `element` in the array that satisfies a provided testing function, or `undefined` if no appropriate element is found. See also `Array.prototype.find()`.

`TypedArray.prototype.findIndex()`

Returns the first index value in the array that has an element that satisfies a provided testing function, or `-1` if no appropriate element was found. See also `Array.prototype.findIndex()`.

`TypedArray.prototype.findLast()`

Returns the value of the last element in the array that satisfies a provided testing function, or `undefined` if no appropriate element is found. See also `Array.prototype.findLast()`.

`TypedArray.prototype.findLastIndex()`

Returns the index of the last element in the array that satisfies a provided testing function, or `-1` if no appropriate element was found. See also `Array.prototype.findLastIndex()`.

`TypedArray.prototype.forEach()`

Calls a function for each element in the array. See also `Array.prototype.forEach()`.

`TypedArray.prototype.includes()`

Determines whether a typed array includes a certain element, returning `true` or `false` as appropriate. See also `Array.prototype.includes()`.

`TypedArray.prototype.indexOf()`

Returns the first (least) index of an element within the array equal to the specified value, or `-1` if none is found. See also `Array.prototype.indexOf()`.

`TypedArray.prototype.join()`

Returns a new string that is the concatenation of all elements in this typed array, separated by commas or a specified separator string. See also `Array.prototype.join()`.

`TypedArray.prototype.keys()`

Returns a new array iterator that contains the keys for each index in the array. See also `Array.prototype.keys()`.

`TypedArray.prototype.lastIndexOf()`

Returns the last (greatest) index of an element within the array equal to the specified value, or `-1` if none is found. See also `Array.prototype.lastIndexOf()`.

`TypedArray.prototype.map()`

Creates a new array with the results of calling a provided function on every element in this array. See also `Array.prototype.map()`.

`TypedArray.prototype.reduce()`

Apply a function against an accumulator and each value of the array (from left-to-right) as to reduce it to a single value. See also `Array.prototype.reduce()`.

`TypedArray.prototype.reduceRight()`

Apply a function against an accumulator and each value of the array (from right-to-left) as to reduce it to a single value. See also `Array.prototype.reduceRight()`.

`TypedArray.prototype.reverse()`

Reverses the order of the elements of an array — the first becomes the last, and the last becomes the first. See also `Array.prototype.reverse()`.

`TypedArray.prototype.set()`

Stores multiple values in the typed array, reading input values from a specified array.

`TypedArray.prototype.slice()`

Extracts a section of an array and returns a new array. See also `Array.prototype.slice()`.

`TypedArray.prototype.some()`

Returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`. See also `Array.prototype.some()`.

`TypedArray.prototype.sort()`

Sorts the elements of an array in place and returns the array. See also `Array.prototype.sort()`.

`TypedArray.prototype.subarray()`

Returns a new `TypedArray` from the given start and end element index.

`TypedArray.prototype.toLocaleString()`

Returns a localized string representing the array and its elements. See also `Array.prototype.toLocaleString()`.

`TypedArray.prototype.toReversed()`

Returns a new array with the elements in reversed order, without modifying the original array.

`TypedArray.prototype.toSorted()`

Returns a new array with the elements sorted in ascending order, without modifying the original array.

`TypedArray.prototype.toString()`

Returns a string representing the array and its elements. See also `Array.prototype.toString()`.

`TypedArray.prototype.values()`

Returns a new *array iterator* object that contains the values for each index in the array. See also `Array.prototype.values()`.

`TypedArray.prototype.with()`

Returns a new array with the element at the given index replaced with the given value, without modifying the original array.

`TypedArray.prototype[Symbol.iterator]()`

Returns a new *array iterator* object that contains the values for each index in the array.

Examples

Property access

You can reference elements in the array using standard array index syntax (that is, using bracket notation). However, getting or setting indexed properties on typed arrays will not search in the prototype chain for this property, even when the indices are out of bound. Indexed properties will consult the `ArrayBuffer` and will never look at object properties. You can still use named properties, just like with all objects.

```js
// Setting and getting using standard array syntax
const int16 = new Int16Array(2);
int16[0] = 42;
console.log(int16[0]); // 42
```

(code continues on the next page)

(code continued from the previous page)

```
// Indexed properties on prototypes are not consulted (Fx 25)
Int8Array.prototype[20] = "foo";
new Int8Array(32)[20]; // 0
// even when out of bound
Int8Array.prototype[20] = "foo";
new Int8Array(8)[20]; // undefined
// or with negative integers
Int8Array.prototype[-1] = "foo";
new Int8Array(8)[-1]; // undefined

// Named properties are allowed, though (Fx 30)
Int8Array.prototype.foo = "bar";
new Int8Array(32).foo; // "bar"
```

Cannot be frozen

`TypedArrays` that aren't empty cannot be frozen, as their underlying `ArrayBuffer` could be mutated through another `TypedArray` view of the buffer. This would mean that the object would never genuinely be frozen.

js · ✗ bad example

```
const i8 = Int8Array.of(1, 2, 3);
Object.freeze(i8);
// TypeError: Cannot freeze array buffer views with elements
```

ByteOffset must be aligned

When constructing a `TypedArray` as a view onto an `ArrayBuffer`, the `byteOffset` argument must be aligned to its element size; in other words, the offset must be a multiple of `BYTES_PER_ELEMENT`.

js · ✗ bad example

```
const i32 = new Int32Array(new ArrayBuffer(4), 1);
// RangeError: start offset of Int32Array should be a multiple of 4
```

js · ✓ good example

```
const i32 = new Int32Array(new ArrayBuffer(4), 0);
```

ByteLength must be aligned

Like the `byteOffset` parameter, the `byteLength` property of an `ArrayBuffer` passed to a `TypedArray`'s constructor must be a multiple of the constructor's `BYTES_PER_ELEMENT`.

js · ✗ bad example

```
const i32 = new Int32Array(new ArrayBuffer(3));
// RangeError: byte length of Int32Array should be a multiple of 4
```

js · ✓ good example

```
const i32 = new Int32Array(new ArrayBuffer(4));
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`TypedArray`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
[Symbol.iterator]	38	12	36[1,2,3,4]	25	10	38	36[1,2,3,4]	25	10	3.0	38	10	1.0.0	1.0	0.12.0
[Symbol.species]	51	13	48	38	10	51	48	41	10	5.0	51	10	1.0.0	1.0	6.5.0
`BYTES_PER_ELEMENT`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`at`	92	92	90	78	15.4	92	90	65	15.4	16.0	92	15.4	1.0.0	**No**	16.6.0
`buffer`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`byteLength`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`byteOffset`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`copyWithin`	45	12	34	32	10	45	34	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`entries`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	0.12.0
`every`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`fill`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`filter`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`find`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`findIndex`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`findLast`	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0
`findLastIndex`	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0
`forEach`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`from`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`includes`	47	14	43	34	10	47	43	34	10	5.0	47	10	1.0.0	1.0	6.0.0
`indexOf`	45	12	37[5]	32	10	45	37[6]	32	10	5.0	45	10	1.0.0	1.0	4.0.0
Indexed properties not consulting prototype	7[7]	12[7]	25	11.6[7]	5.1[7]	18[7]	25	12[7]	5[7]	1.0[7]	4.4[7]	5[7]	1.0.0[7]	1.0[7]	0.10.0[7]
Iterable in constructor	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`join`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`keys`	38	12	37	25	10	38	37	25	10	3.0	38	10	1.0.0	1.0	0.12.0
`lastIndexOf`	45	12	37[8]	32	10	45	37[9]	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`length`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`map`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`name`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Named properties	7	12	30	11.6	5.1	18	30	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`of`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`reduce`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`reduceRight`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`reverse`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`set`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`slice`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`some`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0
`sort`	45	12	46	32	10	45	46	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`subarray`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.12.0
`toLocaleString`	7	12	51	11.6	5.1	18	51	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`toReversed`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0
`toSorted`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0
`toString`	7	12	51	11.6	5.1	18	51	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`values`	38	12	37	25	10	38	37	25	10	3.0	38	10	1.0.0	1.0	0.12.0
`with`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.
5. Starting with Firefox 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).indexOf(0, -0)` will now always return `+0`.
6. Starting with Firefox for Android 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).indexOf(0, -0)` will now always return `+0`.
7. Negative integers are not considered as indexed properties and therefore return the value of the prototype property.
8. Starting with Firefox 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).lastIndexOf(0, -0)` will now always return `+0`.
9. Starting with Firefox for Android 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).lastIndexOf(0, -0)` will now always return `+0`.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of typed arrays in `core-js`
- JavaScript typed arrays guide
- `ArrayBuffer`
- `DataView`
- `TextDecoder`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/from · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`TypedArray.from()`** static method creates a new typed array from an array-like or iterable object. This method is nearly the same as `Array.from()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.from()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uint16 = Int16Array.from("12345");

console.log(uint16);
// Expected output: Int16Array [1, 2, 3, 4, 5]
```

Syntax

js

```
TypedArray.from(arrayLike, mapFn)
TypedArray.from(arrayLike, mapFn, thisArg)
```

Where `TypedArray` is one of:

- `Int8Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `Int16Array`
- `Uint16Array`
- `Int32Array`
- `Uint32Array`
- `Float16Array`
- `Float32Array`
- `Float64Array`
- `BigInt64Array`
- `BigUint64Array`

Parameters

`arrayLike`

An iterable or array-like object to convert to a typed array.

`mapFn` Optional

A function to call on every element of the typed array. If provided, every value to be added to the array is first passed through this function, and `mapFn`'s return value is added to the typed array instead. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`thisArg` Optional

Value to use as `this` when executing `mapFn`.

Return value

A new TypedArray instance.

Description

See Array.from() for more details.

There are some subtle distinctions between Array.from() and TypedArray.from() (note: the this value mentioned below is the this value that TypedArray.from() was called with, not the thisArg argument used to invoke mapFn):

- If the this value of TypedArray.from() is not a constructor, TypedArray.from() will throw a TypeError, while Array.from() defaults to creating a new Array.
- The object constructed by this must be a TypedArray instance, while Array.from() allows its this value to be constructed to any object.
- When the source parameter is an iterator, TypedArray.from() first collects all the values from the iterator, then creates an instance of this using the count, and finally sets the values on the instance. Array.from() sets each value as it receives them from the iterator, then sets its length at the end.
- TypedArray.from() uses [[Set]] while Array.from() uses [[DefineOwnProperty]]. Hence, when working with Proxy objects, it calls handler.set() to create new elements rather than handler.defineProperty().
- When Array.from() gets an array-like which isn't an iterator, it respects holes. TypedArray.from() will ensure the result is dense.

Examples

From an iterable object (Set)

```js
const s = new Set([1, 2, 3]);
Uint8Array.from(s);
// Uint8Array [ 1, 2, 3 ]
```

From a string

```js
Int16Array.from("123");
// Int16Array [ 1, 2, 3 ]
```

Use with arrow function and map

Using an arrow function as the map function to manipulate the elements

```js
Float32Array.from([1, 2, 3], (x) => x + x);
// Float32Array [ 2, 4, 6 ]
```

Generate a sequence of numbers

```js
Uint8Array.from({ length: 5 }, (v, k) => k);
// Uint8Array [ 0, 1, 2, 3, 4 ]
```

Calling from() on non-TypedArray constructors

The `this` value of `from()` must be a constructor that returns a `TypedArray` instance.

```js
function NotArray(len) {
  console.log("NotArray called with length", len);
}

Int8Array.from.call({}, []); // TypeError: #<Object> is not a constructor
Int8Array.from.call(NotArray, []);
// NotArray called with length 0
// TypeError: Method %TypedArray%.from called on incompatible receiver #<NotArray>
```

```js
function NotArray2(len) {
  console.log("NotArray2 called with length", len);
  return new Uint8Array(len);
}
console.log(Int8Array.from.call(NotArray2, [1, 2, 3]));
// NotArray2 called with length 3
// Uint8Array(3) [ 1, 2, 3 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.from`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.from` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.of()`
- `TypedArray.prototype.map()`
- `Array.from()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.of()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/of · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`TypedArray.of()`** static method creates a new typed array from a variable number of arguments. This method is nearly the same as `Array.of()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.of()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const int16array = Int16Array.of("10", "20", "30", "40", "50");

console.log(int16array);
// Expected output: Int16Array [10, 20, 30, 40, 50]
```

Syntax

js

```
TypedArray.of()
TypedArray.of(element1)
TypedArray.of(element1, element2)
TypedArray.of(element1, element2, /* …, */ elementN)
```

Where `TypedArray` is one of:

- `Int8Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `Int16Array`
- `Uint16Array`
- `Int32Array`
- `Uint32Array`
- `Float16Array`
- `Float32Array`
- `Float64Array`
- `BigInt64Array`
- `BigUint64Array`

Parameters

`element1, ..., elementN`

Elements used to create the typed array.

Return value

A new `TypedArray` instance.

Description

See `Array.of()` for more details. There are some subtle distinctions between `Array.of()` and `TypedArray.of()`:

- If the `this` value passed to `TypedArray.of()` is not a constructor, `TypedArray.of()` will throw a `TypeError`, while `Array.of()` defaults to creating a new `Array`.
- `TypedArray.of()` uses `[[Set]]` while `Array.of()` uses `[[DefineOwnProperty]]`. Hence, when working with `Proxy` objects, it calls `handler.set()` to create new elements rather than `handler.defineProperty()`.

Examples

Using of()

```js
Uint8Array.of(1); // Uint8Array [ 1 ]
Int8Array.of("1", "2", "3"); // Int8Array [ 1, 2, 3 ]
Float32Array.of(1, 2, 3); // Float32Array [ 1, 2, 3 ]
Int16Array.of(undefined); // Int16Array [ 0 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.of`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`of`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.of`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.of` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.from()`
- `Array.of()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.BYTES_PER_ELEMENT

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/BYTES_PER_ELEMENT · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`TypedArray.BYTES_PER_ELEMENT`** static data property represents the size in bytes of each element in a typed array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.BYTES_PER_ELEMENT". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(Float64Array.BYTES_PER_ELEMENT);
// Expected output: 8

console.log(Int8Array.BYTES_PER_ELEMENT);
// Expected output: 1
```

Value

A number whose value depends on the type of `TypedArray`.

Property attributes of `TypedArray.BYTES_PER_ELEMENT`	
Writable	no
Enumerable	no
Configurable	no

Description

`TypedArray` objects differ from each other in the number of bytes per element and in the way the bytes are interpreted. The `BYTES_PER_ELEMENT` constant contains the number of bytes each element in the given `TypedArray` has.

The `BYTES_PER_ELEMENT` property is both an *instance property* and a *static property*. It's available on both `TypedArray` subclass constructors and on instances of those constructors.

As an instance property, `BYTES_PER_ELEMENT` is defined on the constructor's `prototype`.

js

```
console.log(Object.hasOwn(Int8Array.prototype, "BYTES_PER_ELEMENT")); // true
```

Examples

Using BYTES_PER_ELEMENT

As a static property:

```js
Int8Array.BYTES_PER_ELEMENT; // 1
Uint8Array.BYTES_PER_ELEMENT; // 1
Uint8ClampedArray.BYTES_PER_ELEMENT; // 1
Int16Array.BYTES_PER_ELEMENT; // 2
Uint16Array.BYTES_PER_ELEMENT; // 2
Float16Array.BYTES_PER_ELEMENT; // 2
Int32Array.BYTES_PER_ELEMENT; // 4
Uint32Array.BYTES_PER_ELEMENT; // 4
Float32Array.BYTES_PER_ELEMENT; // 4
Float64Array.BYTES_PER_ELEMENT; // 8
BigInt64Array.BYTES_PER_ELEMENT; // 8
BigUint64Array.BYTES_PER_ELEMENT; // 8
```

As an instance property:

```js
new Int8Array([]).BYTES_PER_ELEMENT; // 1
new Uint8Array([]).BYTES_PER_ELEMENT; // 1
new Uint8ClampedArray([]).BYTES_PER_ELEMENT; // 1
new Int16Array([]).BYTES_PER_ELEMENT; // 2
new Uint16Array([]).BYTES_PER_ELEMENT; // 2
new Float16Array([]).BYTES_PER_ELEMENT; // 2
new Int32Array([]).BYTES_PER_ELEMENT; // 4
new Uint32Array([]).BYTES_PER_ELEMENT; // 4
new Float32Array([]).BYTES_PER_ELEMENT; // 4
new Float64Array([]).BYTES_PER_ELEMENT; // 8
new BigInt64Array([]).BYTES_PER_ELEMENT; // 8
new BigUint64Array([]).BYTES_PER_ELEMENT; // 8
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray.bytes_per_element`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`BYTES_PER_ELEMENT`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.BYTES_PER_ELEMENT`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`

TypedArray[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/Symbol.species · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`TypedArray[Symbol.species]`** static accessor property returns the constructor used to construct return values from typed array methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
TypedArray[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from typed array methods that create new typed arrays.

Description

The `[Symbol.species]` accessor property returns the default constructor for typed array objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```
// Hypothetical underlying implementation for illustration
class TypedArray {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```
class SubTypedArray extends Int8Array {}
SubTypedArray[Symbol.species] === SubTypedArray; // true
```

When calling typed array methods that do not mutate the existing array but return a new array instance (for example, `filter()` and `map()`), the array's `constructor[Symbol.species]` will be accessed. The returned constructor will be used to construct the return value of the typed array method.

However, unlike `Array[Symbol.species]`, when using `[Symbol.species]` to create new typed arrays, the language will make sure that the newly created array is a proper typed array and has the same content type as the original array — for example, you can't create a `BigInt64Array` from a `Float64Array`, or create a non-BigInt array from a BigInt array. Doing so throws a `TypeError`.

```js
class BadArray extends Int8Array {
  static get [Symbol.species]() {
    return Array;
  }
}
new BadArray(1).map(() => 0); // TypeError: Method %TypedArray%.prototype.map called on
incompatible receiver [object Array]

class BadArray2 extends Int8Array {
  static get [Symbol.species]() {
    return BigInt64Array;
  }
}
new BadArray2(1).map(() => 0n); // TypeError: TypedArray.prototype.map constructed typed
array of different content type from |this|
```

> **Note:** Due to a bug in both SpiderMonkey and V8, the content type match is not checked. Only Safari will throw a `TypeError` in the second example.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is one of the typed array constructors itself for any given typed array constructor.

```js
Int8Array[Symbol.species]; // function Int8Array()
Uint8Array[Symbol.species]; // function Uint8Array()
Float32Array[Symbol.species]; // function Float32Array()
```

Species in derived objects

In an instance of a custom `TypedArray` subclass, such as `MyTypedArray`, the `MyTypedArray` species is the `MyTypedArray` constructor. However, you might want to overwrite this, in order to return a parent typed array object in your derived class methods:

```js
class MyTypedArray extends Uint8Array {
  // Overwrite MyTypedArray species to the parent Uint8Array constructor
  static get [Symbol.species]() {
    return Uint8Array;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-get-%typedarray%-%symbol.species%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.species]	51	13	48	38	10	51	48	41	10	5.0	51	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.@@species`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `TypedArray`
- `Symbol.species`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.at()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/at · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since March 2022.

The **`at()`** method of `TypedArray` instances takes an integer value and returns the item at that index, allowing for positive and negative integers. Negative integers count back from the last item in the typed array. This method has the same algorithm as `Array.prototype.at()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.at()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const int8 = new Int8Array([0, 10, -10, 20, -30, 40, -50]);

let index = 1;

console.log(`An index of ${index} returns the item ${int8.at(index)}`);
// Expected output: "An index of 1 returns the item 10"

index = -2;

console.log(`An index of ${index} returns the item ${int8.at(index)}`);
// Expected output: "An index of -2 returns the item 40"
```

Syntax

js

```
at(index)
```

Parameters

`index`

Zero-based index of the typed array element to be returned, converted to an integer. Negative index counts back from the end of the typed array — if `index < 0`, `index + array.length` is accessed.

Return value

The element in the typed array matching the given index. Always returns `undefined` if `index < -array.length` or `index >= array.length` without attempting to access the corresponding property.

Description

See `Array.prototype.at()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Return the last value of a typed array

The following example provides a function which returns the last element found in a specified array.

```js
const uint8 = new Uint8Array([1, 2, 4, 7, 11, 18]);

// A function which returns the last item of a given array
function returnLast(arr) {
  return arr.at(-1);
}

const lastItem = returnLast(uint8);
console.log(lastItem); // 18
```

Comparing methods

Here we compare different ways to select the penultimate (last but one) item of a `TypedArray`. While all below methods are valid, it highlights the succinctness and readability of the `at()` method.

```js
// Our typed array with values
const uint8 = new Uint8Array([1, 2, 4, 7, 11, 18]);

// Using length property
const lengthWay = uint8[uint8.length - 2];
console.log(lengthWay); // 11

// Using slice() method. Note an array is returned
const sliceWay = uint8.slice(-2, -1);
console.log(sliceWay[0]); // 11

// Using at() method
const atWay = uint8.at(-2);
console.log(atWay); // 11
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.at`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`at`	92	92	90	78	15.4	92	90	65	15.4	16.0	92	15.4	1.0.0	**No**	16.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.at`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.at` in `core-js`
- es-shims polyfill of `TypedArray.prototype.at`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.indexOf()`
- `TypedArray.prototype.with()`
- `Array.prototype.at()`
- `String.prototype.at()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.copyWithin()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/copyWithin · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`copyWithin()`** method of `TypedArray` instances shallow copies part of this typed array to another location in the same typed array and returns this typed array without modifying its length. This method has the same algorithm as `Array.prototype.copyWithin()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.copyWithin()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([1, 2, 3, 4, 5, 6, 7, 8]);

// Insert position, start position, end position
uint8.copyWithin(3, 1, 3);

console.log(uint8);
// Expected output: Uint8Array [1, 2, 3, 2, 3, 6, 7, 8]
```

Syntax

js

```
copyWithin(target, start)
copyWithin(target, start, end)
```

Parameters

`target`

Zero-based index at which to copy the sequence to, converted to an integer. This corresponds to where the element at `start` will be copied to, and all elements between `start` and `end` are copied to succeeding indices.

`start`

Zero-based index at which to start copying elements from, converted to an integer.

`end` Optional

Zero-based index at which to end copying elements from, converted to an integer. `copyWithin()` copies up to but not including `end`.

Return value

The modified typed array.

Description

See `Array.prototype.copyWithin()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using copyWithin()

js

```
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);
uint8.set([1, 2, 3]);
console.log(uint8); // Uint8Array [ 1, 2, 3, 0, 0, 0, 0, 0 ]
uint8.copyWithin(3, 0, 3);
console.log(uint8); // Uint8Array [ 1, 2, 3, 1, 2, 3, 0, 0 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.copywithin`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`copyWithin`	45	12	34	32	10	45	34	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.copyWithin`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.copyWithin` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `Array.prototype.copyWithin()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.entries()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/entries · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`entries()`** method of `TypedArray` instances returns a new *array iterator* object that contains the key/value pairs for each index in the typed array. This method has the same algorithm as `Array.prototype.entries()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.entries()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);
const eArr = uint8.entries();

eArr.next();
eArr.next();

console.log(eArr.next().value);
// Expected output: Array [2, 30]
```

Syntax

```js
entries()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

See `Array.prototype.entries()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Iteration using for...of loop

```js
const array = new Uint8Array([10, 20, 30, 40, 50]);
const arrayEntries = arr.entries();
for (const element of arrayEntries) {
  console.log(element);
}
```

Alternative iteration

```js
const array = new Uint8Array([10, 20, 30, 40, 50]);
const arrayEntries = arr.entries();

console.log(arrayEntries.next().value); // [0, 10]
console.log(arrayEntries.next().value); // [1, 20]
console.log(arrayEntries.next().value); // [2, 30]
console.log(arrayEntries.next().value); // [3, 40]
console.log(arrayEntries.next().value); // [4, 50]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.entries

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`entries`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.entries`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.entries` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.keys()`
- `TypedArray.prototype.values()`
- `TypedArray.prototype[Symbol.iterator]()`
- `Array.prototype.entries()`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.every()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/every · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`every()`** method of `TypedArray` instances returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`. This method has the same algorithm as `Array.prototype.every()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.every()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([-10, -20, -30, -40, -50]);

console.log(int8.every(isNegative));
// Expected output: true
```

Syntax

```js
every(callbackFn)
every(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `every()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`true` unless `callbackFn` returns a falsy value for a typed array element, in which case `false` is immediately returned.

Description

See `Array.prototype.every()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Testing size of all typed array elements

The following example tests whether all elements in the typed array are 10 or bigger.

```js
function isBigEnough(element, index, array) {
  return element >= 10;
}
new Uint8Array([12, 5, 8, 130, 44]).every(isBigEnough); // false
new Uint8Array([12, 54, 18, 130, 44]).every(isBigEnough); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.every

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
every	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.every`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.every` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.forEach()`
- `TypedArray.prototype.some()`
- `TypedArray.prototype.find()`
- `Array.prototype.every()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.fill()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/fill · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`fill()`** method of `TypedArray` instances changes all elements within a range of indices in a typed array to a static value. It returns the modified typed array. This method has the same algorithm as `Array.prototype.fill()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.fill()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([0, 0, 0, 0]);
// Value, start position, end position
uint8.fill(4, 1, 3);

console.log(uint8);
// Expected output: Uint8Array [0, 4, 4, 0]
```

Syntax

js

```
fill(value)
fill(value, start)
fill(value, start, end)
```

Parameters

`value`

Value to fill the typed array with.

`start` Optional

Zero-based index at which to start filling, converted to an integer.

`end` Optional

Zero-based index at which to end filling, converted to an integer. `fill()` fills up to but not including `end`.

Return value

The modified typed array, filled with `value`.

Description

See `Array.prototype.fill()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using fill()

```js
new Uint8Array([1, 2, 3]).fill(4); // Uint8Array [4, 4, 4]
new Uint8Array([1, 2, 3]).fill(4, 1); // Uint8Array [1, 4, 4]
new Uint8Array([1, 2, 3]).fill(4, 1, 2); // Uint8Array [1, 4, 3]
new Uint8Array([1, 2, 3]).fill(4, 1, 1); // Uint8Array [1, 2, 3]
new Uint8Array([1, 2, 3]).fill(4, -3, -2); // Uint8Array [4, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.fill`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fill`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.fill`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.prototype.fill` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `Array.prototype.fill()`

TypedArray.prototype.filter()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/filter · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`filter()`** method of `TypedArray` instances creates a copy of a portion of a given typed array, filtered down to just the elements from the given typed array that pass the test implemented by the provided function. This method has the same algorithm as `Array.prototype.filter()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.filter()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([-10, 20, -30, 40, -50]);
const negInt8 = int8.filter(isNegative);

console.log(negInt8);
// Expected output: Int8Array [-10, -30, -50]
```

Syntax

js

```
filter(callbackFn)
filter(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to keep the element in the resulting typed array, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `filter()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A copy of the given typed array containing just the elements that pass the test. If no elements pass the test, an empty typed array is returned.

Description

See `Array.prototype.filter()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Filtering out all small values

The following example uses `filter()` to create a filtered typed array that has all elements with values less than 10 removed.

```js
function isBigEnough(element, index, array) {
  return element >= 10;
}
new Uint8Array([12, 5, 8, 130, 44]).filter(isBigEnough);
// Uint8Array [ 12, 130, 44 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.filter`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`filter`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.filter`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.prototype.filter` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.forEach()`
- `TypedArray.prototype.every()`

- TypedArray.prototype.map()
- TypedArray.prototype.some()
- TypedArray.prototype.reduce()
- Array.prototype.filter()

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.find()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/find · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`find()`** method of TypedArray instances returns the first element in the provided typed array that satisfies the provided testing function. If no values satisfy the testing function, undefined is returned. This method has the same algorithm as `Array.prototype.find()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.find()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([10, 0, -10, 20, -30, 40, -50]);

console.log(int8.find(isNegative));
// Expected output: -10
```

Syntax

js

```
find(callbackFn)
find(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `find()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The first element in the typed array that satisfies the provided testing function. Otherwise, `undefined` is returned.

Description

See `Array.prototype.find()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the first prime number in a typed array

The following example returns the first element in the typed array that is a prime number, or `undefined` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

const uint8 = new Uint8Array([4, 5, 8, 12]);
console.log(uint8.find(isPrime)); // 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.find

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`find`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.find`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.prototype.find` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLast()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.includes()`
- `TypedArray.prototype.filter()`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.some()`
- `Array.prototype.find()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.findIndex()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/findIndex · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`findIndex()`** method of `TypedArray` instances returns the index of the first element in a typed array that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned. This method has the same algorithm as `Array.prototype.findIndex()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.findIndex()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([10, -20, 30, -40, 50]);

console.log(int8.findIndex(isNegative));
// Expected output: 1
```

Syntax

```js
findIndex(callbackFn)
findIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `findIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the first element in the typed array that passes the test. Otherwise, `-1`.

Description

See `Array.prototype.findIndex()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the index of the first prime number in a typed array

The following example returns the index of the first element in the typed array that is a prime number, or `-1` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}
```

(code continues on the next page)

(code continued from the previous page)

```
const uint8 = new Uint8Array([4, 6, 8, 12]);
const uint16 = new Uint16Array([4, 6, 7, 12]);

console.log(uint8.findIndex(isPrime)); // -1, not found
console.log(uint16.findIndex(isPrime)); // 2
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.findindex`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`findIndex`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.findIndex`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.findIndex` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findLast()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.indexOf()`
- `TypedArray.prototype.lastIndexOf()`
- `Array.prototype.findIndex()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.findLast()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/findLast · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since August 2022.

The **`findLast()`** method of `TypedArray` instances iterates the typed array in reverse order and returns the value of the first element that satisfies the provided testing function. If no elements satisfy the testing function, `undefined` is returned. This method has the same algorithm as `Array.prototype.findLast()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.findLast()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function isNegative(element /*, index, array */) {
  return element < 0;
}

const int8 = new Int8Array([10, 0, -10, 20, -30, 40, 50]);

console.log(int8.findLast(isNegative));
// Expected output: -30
```

Syntax

js

```
findLast(callbackFn)
findLast(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `findLast()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The last (highest-index) element in the typed array that satisfies the provided testing function; `undefined` if no matching element is found.

Description

See `Array.prototype.findLast()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the last prime number in a typed array

The following example returns the last element in the typed array that is a prime number, or `undefined` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

let uint8 = new Uint8Array([4, 6, 8, 12]);
console.log(uint8.findLast(isPrime)); // undefined (no primes in array)
uint8 = new Uint8Array([4, 5, 7, 8, 9, 11, 12]);
console.log(uint8.findLast(isPrime)); // 11
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.findlast`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`findLast`	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.findLast`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.findLast` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.includes()`
- `TypedArray.prototype.filter()`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.some()`
- `Array.prototype.findLast()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.findLastIndex()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/findLastIndex · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since August 2022.

The **`findLastIndex()`** method of `TypedArray` instances iterates the typed array in reverse order and returns the index of the first element that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned. This method has the same algorithm as `Array.prototype.findLastIndex()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.findLastIndex()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function isNegative(element /*, index, array */) {
  return element < 0;
}

const int8 = new Int8Array([10, -20, 30, -40, 50]);

console.log(int8.findLastIndex(isNegative));
// Expected output: 3
```

Syntax

js

```
findLastIndex(callbackFn)
findLastIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `findLastIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the last (highest-index) element in the typed array that passes the test. Otherwise `-1` if no matching element is found.

Description

See `Array.prototype.findLastIndex()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Find the index of the last prime number in a typed array

The following example returns the index of the last element in the typed array that is a prime number, or `-1` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

let uint8 = new Uint8Array([4, 6, 8, 12]);
console.log(uint8.findLastIndex(isPrime));
// -1 (no primes in array)
uint8 = new Uint8Array([4, 5, 7, 8, 9, 11, 12]);
console.log(uint8.findLastIndex(isPrime));
// 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.findlastindex`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`findLastIndex`	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.findLastIndex`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.findLastIndex` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findIndex()`

- TypedArray.prototype.findLast()
- TypedArray.prototype.indexOf()
- TypedArray.prototype.lastIndexOf()
- Array.prototype.findLastIndex()

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.forEach()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/forEach · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`forEach()`** method of TypedArray instances executes a provided function once for each typed array element. This method has the same algorithm as Array.prototype.forEach().

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.forEach()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30]);

uint8.forEach((element) => console.log(element));

// Expected output: 10
// Expected output: 20
// Expected output: 30
```

Syntax

js

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value is discarded. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `forEach()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

None (`undefined`).

Description

See `Array.prototype.forEach()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Logging the contents of a typed array

The following code logs a line for each element in a typed array:

```js
function logArrayElements(element, index, array) {
  console.log(`a[${index}] = ${element}`);
}

new Uint8Array([0, 1, 2, 3]).forEach(logArrayElements);
// Logs:
// a[0] = 0
// a[1] = 1
// a[2] = 2
// a[3] = 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.foreach`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`forEach`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.forEach`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.prototype.forEach` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.map()`

- TypedArray.prototype.filter()
- TypedArray.prototype.every()
- TypedArray.prototype.some()
- Array.prototype.forEach()
- Map.prototype.forEach()
- Set.prototype.forEach()

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.includes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/includes · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`includes()`** method of TypedArray instances determines whether a typed array includes a certain value among its entries, returning `true` or `false` as appropriate. This method has the same algorithm as Array.prototype.includes().

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.includes()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.includes(20));
// Expected output: true

// Check from position 3
console.log(uint8.includes(20, 3));
// Expected output: false
```

Syntax

js

```
includes(searchElement)
includes(searchElement, fromIndex)
```

Parameters

`searchElement`

The value to search for.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

Return value

A boolean value which is `true` if the value `searchElement` is found within the typed array (or the part of the typed array indicated by the index `fromIndex`, if specified).

Description

See `Array.prototype.includes()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using includes()

```js
const uint8 = new Uint8Array([1, 2, 3]);
uint8.includes(2); // true
uint8.includes(4); // false
uint8.includes(3, 3); // false

// NaN handling (only relevant for floating point arrays)
new Uint8Array([NaN]).includes(NaN); // false, since the NaN passed to the constructor gets converted to 0
new Float32Array([NaN]).includes(NaN); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.includes`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`includes`	47	14	43	34	10	47	43	34	10	5.0	47	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.includes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.includes` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.indexOf()`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.findIndex()`
- `Array.prototype.includes()`
- `String.prototype.includes()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.indexOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/indexOf · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`indexOf()`** method of `TypedArray` instances returns the first index at which a given element can be found in the typed array, or -1 if it is not present. This method has the same algorithm as `Array.prototype.indexOf()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.indexOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.indexOf(50));
// Expected output: 4

// From position 3
console.log(uint8.indexOf(20, 3));
// Expected output: -1

console.log(uint8.indexOf(51));
// Expected output: -1
```

Syntax

js

```
indexOf(searchElement)
indexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the typed array.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

Return value

The first index of `searchElement` in the typed array; `-1` if not found.

Description

See `Array.prototype.indexOf()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using indexOf()

```js
const uint8 = new Uint8Array([2, 5, 9]);
uint8.indexOf(2); // 0
uint8.indexOf(7); // -1
uint8.indexOf(9, 2); // 2
uint8.indexOf(2, -1); // -1
uint8.indexOf(2, -3); // 0
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.indexof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
indexOf	45	12	37[1]	32	10	45	37[2]	32	10	5.0	45	10	1.0.0	1.0	4.0.0

1. Starting with Firefox 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).indexOf(0, -0)` will now always return `+0`.
2. Starting with Firefox for Android 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).indexOf(0, -0)` will now always return `+0`.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.indexOf`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.indexOf` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.lastIndexOf()`
- `Array.prototype.indexOf()`
- `String.prototype.indexOf()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.join()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/join · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`join()`** method of `TypedArray` instances returns a new string that is the concatenation of all elements in this typed array, separated by commas or a specified separator string. If the typed array has only one item, that item's stringification is returned without using the separator. This method has the same algorithm as `Array.prototype.join()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.join()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.join());
// Expected output: "10,20,30,40,50"

console.log(uint8.join(""));
// Expected output: "1020304050"

console.log(uint8.join("-"));
// Expected output: "10-20-30-40-50"
```

Syntax

js

```
join()
join(separator)
```

Parameters

`separator` Optional

A string to separate each pair of adjacent elements of the typed array. If omitted, the typed array elements are separated with a comma (",").

Return value

A string with all typed array elements joined. If `array.length` is `0`, the empty string is returned.

Description

See `Array.prototype.join()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using join()

```js
const uint8 = new Uint8Array([1, 2, 3]);
uint8.join(); // '1,2,3'
uint8.join(" / "); // '1 / 2 / 3'
uint8.join(""); // '123'
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.join

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
join	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.join`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.join` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.toString()`
- `Array.prototype.join()`
- `String.prototype.split()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.keys()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/keys · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`keys()`** method of `TypedArray` instances returns a new *array iterator* object that contains the keys for each index in the typed array. This method has the same algorithm as `Array.prototype.keys()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.keys()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);
const keys = uint8.keys();

keys.next();
keys.next();

console.log(keys.next().value);
// Expected output: 2
```

Syntax

js

```
keys()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

See `Array.prototype.keys()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Iteration using for...of loop

js

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const arrKeys = arr.keys();
for (const n of arrKeys) {
  console.log(n);
}
```

Alternative iteration

js

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const arrKeys = arr.keys();
console.log(arrKeys.next().value); // 0
console.log(arrKeys.next().value); // 1
console.log(arrKeys.next().value); // 2
console.log(arrKeys.next().value); // 3
console.log(arrKeys.next().value); // 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.keys`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
keys	38	12	37	25	10	38	37	25	10	3.0	38	10	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.keys`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.keys` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.entries()`
- `TypedArray.prototype.values()`
- `TypedArray.prototype[Symbol.iterator]()`
- `Array.prototype.keys()`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.lastIndexOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/lastIndexOf · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`lastIndexOf()`** method of `TypedArray` instances returns the last index at which a given element can be found in the typed array, or -1 if it is not present. The typed array is searched backwards, starting at `fromIndex`. This method has the same algorithm as `Array.prototype.lastIndexOf()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.lastIndexOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 50, 50, 50, 60]);

console.log(uint8.lastIndexOf(50, 5));
// Expected output: 4

console.log(uint8.lastIndexOf(50, 3));
// Expected output: 3
```

Syntax

js

```
lastIndexOf(searchElement)
lastIndexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the typed array.

`fromIndex` Optional

Zero-based index at which to start searching backwards, converted to an integer.

Return value

The last index of `searchElement` in the typed array; `-1` if not found.

Description

See `Array.prototype.lastIndexOf()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using lastIndexOf()

js

```
const uint8 = new Uint8Array([2, 5, 9, 2]);
uint8.lastIndexOf(2); // 3
uint8.lastIndexOf(7); // -1
uint8.lastIndexOf(2, 3); // 3
uint8.lastIndexOf(2, 2); // 0
uint8.lastIndexOf(2, -2); // 0
uint8.lastIndexOf(2, -1); // 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.lastindexof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`lastIndexOf`	45	12	37[1]	32	10	45	37[2]	32	10	5.0	45	10	1.0.0	1.0	4.0.0

1. Starting with Firefox 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).lastIndexOf(0, -0)` will now always return `+0`.
2. Starting with Firefox for Android 47, this method will no longer return `-0`. For example, `new Uint8Array([0]).lastIndexOf(0, -0)` will now always return `+0`.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.lastIndexOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.lastIndexOf` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.findIndex()`
- `TypedArray.prototype.findLastIndex()`
- `TypedArray.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`
- `String.prototype.lastIndexOf()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.map()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/map · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`map()`** method of `TypedArray` instances creates a new typed array populated with the results of calling a provided function on every element in the calling typed array. This method has the same algorithm as `Array.prototype.map()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.map()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([25, 36, 49]);
const roots = uint8.map(Math.sqrt);

console.log(roots);
// Expected output: Uint8Array [5, 6, 7]
```

Syntax

```js
map(callbackFn)
map(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value is added as a single element in the new typed array. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `map()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A new typed array with each element being the result of the callback function.

Description

See `Array.prototype.map()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Mapping a typed array to a typed array of square roots

The following code takes a typed array and creates a new typed array containing the square roots of the numbers in the first typed array.

```js
const numbers = new Uint8Array([1, 4, 9]);
const roots = numbers.map(Math.sqrt);
// roots is now: Uint8Array [1, 2, 3],
// numbers is still Uint8Array [1, 4, 9]
```

Mapping a typed array of numbers using a function containing an argument

The following code shows how `map()` works when a function requiring one argument is used with it. The argument will automatically be assigned to each element of the typed array as `map()` loops through the original typed array.

```js
const numbers = new Uint8Array([1, 4, 9]);
const doubles = numbers.map((num) => num * 2);
// doubles is now Uint8Array [2, 8, 18]
// numbers is still Uint8Array [1, 4, 9]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.map

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
map	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.map`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.map` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.forEach()`
- `TypedArray.from()`
- `Array.prototype.map()`
- `Map`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.reduce()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/reduce · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`reduce()`** method of `TypedArray` instances executes a user-supplied "reducer" callback function on each element of the typed array, in order, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements of the typed array is a single value. This method has the same algorithm as `Array.prototype.reduce()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.reduce()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uint8 = new Uint8Array([0, 1, 2, 3]);

function sum(accumulator, currentValue) {
  return accumulator + currentValue;
}

console.log(uint8.reduce(sum));
// Expected output: 6
```

Syntax

js

```
reduce(callbackFn)
reduce(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduce()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is `array[0]`.

`currentValue`

The value of the current element. On the first call, its value is `array[0]` if `initialValue` is specified; otherwise its value is `array[1]`.

`currentIndex`

The index position of `currentValue` in the typed array. On the first call, its value is `0` if `initialValue` is specified, otherwise `1`.

`array`

The typed array `reduce()` was called upon.

`initialValue` Optional

A value to which `accumulator` is initialized the first time the callback is called. If `initialValue` is specified, `callbackFn` starts executing with the first value in the typed array as `currentValue`. If `initialValue` is *not* specified, `accumulator` is initialized to the first value in the typed array, and `callbackFn` starts executing with the second value in the typed array as `currentValue`. In this case, if the typed array is empty (so that there's no first value to return as `accumulator`), an error is thrown.

Return value

The value that results from running the "reducer" callback function to completion over the entire typed array.

Exceptions

`TypeError`

Thrown if the typed array contains no elements and `initialValue` is not provided.

Description

See `Array.prototype.reduce()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Sum up all values within an array

```js
const total = new Uint8Array([0, 1, 2, 3]).reduce((a, b) => a + b);
// total === 6
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.reduce`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`reduce`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.reduce`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.reduce` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.map()`
- `TypedArray.prototype.reduceRight()`
- `Array.prototype.reduce()`
- `Object.groupBy()`
- `Map.groupBy()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.reduceRight()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/reduceRight · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`reduceRight()`** method of `TypedArray` instances applies a function against an accumulator and each value of the typed array (from right-to-left) to reduce it to a single value. This method has the same algorithm as `Array.prototype.reduceRight()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.reduceRight()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30]);

const result = uint8.reduceRight(
  (accumulator, currentValue) => `${accumulator}, ${currentValue}`,
);

console.log(result);
// Expected output: "30, 20, 10"
```

Syntax

js

```
reduceRight(callbackFn)
reduceRight(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduceRight()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is the last element of the typed array.

`currentValue`

The value of the current element. On the first call, its value is the last element if `initialValue` is specified; otherwise its value is the second-to-last element.

`currentIndex`

The index position of `currentValue` in the typed array. On the first call, its value is `array.length - 1` if `initialValue` is specified, otherwise `array.length - 2`.

`array`

The typed array `reduceRight()` was called upon.

`initialValue` Optional

Value to use as accumulator to the first call of the `callbackFn`. If no initial value is supplied, the last element in the typed array will be used and skipped. Calling `reduceRight()` on an empty typed array without an initial value creates a `TypeError`.

Return value

The value that results from the reduction.

Description

See `Array.prototype.reduceRight()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Sum up all values within an array

```js
const total = new Uint8Array([0, 1, 2, 3]).reduceRight((a, b) => a + b);
// total === 6
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.reduceright`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`reduceRight`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.reduceRight`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.reduceRight` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.map()`
- `TypedArray.prototype.reduce()`
- `Array.prototype.reduceRight()`
- `Object.groupBy()`
- `Map.groupBy()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.reverse()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/reverse · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`reverse()`** method of `TypedArray` instances reverses a typed array *in place* and returns the reference to the same typed array, the first typed array element now becoming the last, and the last typed array element becoming the first. In other words, elements order in the typed array will be turned towards the direction opposite to that previously stated. This method has the same algorithm as `Array.prototype.reverse()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.reverse()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([1, 2, 3]);
uint8.reverse();

console.log(uint8);
// Expected output: Uint8Array [3, 2, 1]
```

Syntax

js

```
reverse()
```

Parameters

None.

Return value

The reference to the original typed array, now reversed. Note that the typed array is reversed *in place*, and no copy is made.

Description

See `Array.prototype.reverse()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using reverse()

js

```
const uint8 = new Uint8Array([1, 2, 3]);
uint8.reverse();

console.log(uint8); // Uint8Array [3, 2, 1]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.reverse`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`reverse`	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.reverse`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.reverse` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.join()`
- `TypedArray.prototype.sort()`
- `TypedArray.prototype.toReversed()`
- `Array.prototype.reverse()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.set()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/set · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`set()`** method of `TypedArray` instances stores multiple values in the typed array, reading input values from a specified array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.set()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);

// Copy the values into the array starting at index 3
uint8.set([1, 2, 3], 3);

console.log(uint8);
// Expected output: Uint8Array [0, 0, 0, 1, 2, 3, 0, 0]
```

Syntax

js

```
set(array)
set(array, targetOffset)

set(typedarray)
set(typedarray, targetOffset)
```

Parameters

`array`

The array from which to copy values. All values from the source array are copied into the target array, unless the length of the source array plus the target offset exceeds the length of the target array, in which case an exception is thrown.

`typedarray`

If the source array is a typed array, the two arrays may share the same underlying `ArrayBuffer`; the JavaScript engine will intelligently **copy** the source range of the buffer to the destination range.

`targetOffset` Optional

The offset into the target array at which to begin writing values from the source array. If this value is omitted, 0 is assumed (that is, the source array will overwrite values in the target array starting at index 0).

Return value

None (`undefined`).

Exceptions

`RangeError`

Thrown in one of the following cases:

- An element will be stored beyond the end of the typed array, either because `targetOffset` is too large or because `array` or `typedarray` is too large.
- `targetOffset` is negative.

Examples

Using set()

```js
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);

uint8.set([1, 2, 3], 3);

console.log(uint8); // Uint8Array [ 0, 0, 0, 1, 2, 3, 0, 0 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.set

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
set	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.set`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.set` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.slice()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/slice · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`slice()`** method of `TypedArray` instances returns a copy of a portion of a typed array into a new typed array object selected from `start` to `end` (`end` not included) where `start` and `end` represent the index of items in that typed array. The original typed array will not be modified. This method has the same algorithm as `Array.prototype.slice()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.slice()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const bytes = new Uint8Array([10, 20, 30, 40, 50]);
const byteSlice = bytes.slice(1, 3);

console.log(byteSlice);
// Expected output: Uint8Array [20, 30]
```

Syntax

js

```
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

`end` Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

Return value

A new typed array containing the extracted elements.

Description

See `Array.prototype.slice()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Return a portion of an existing typed array

js

```
const bytes = new Uint8Array([1, 2, 3]);
bytes.slice(1); // Uint8Array [ 2, 3 ]
bytes.slice(2); // Uint8Array [ 3 ]
bytes.slice(-2); // Uint8Array [ 2, 3 ]
bytes.slice(0, 1); // Uint8Array [ 1 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.slice`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`slice`	45	12	38	32	10	45	38	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.slice`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.slice` in `core-js`
- es-shims polyfill of `TypedArray.prototype.slice`
- JavaScript typed arrays guide
- `TypedArray`
- `Array.prototype.slice()`
- `String.prototype.slice()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.some()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/some · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`some()`** method of `TypedArray` instances returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`. This method has the same algorithm as `Array.prototype.some()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.some()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function isNegative(element, index, array) {
  return element < 0;
}

const int8 = new Int8Array([-10, 20, -30, 40, -50]);
const positives = new Int8Array([10, 20, 30, 40, 50]);

console.log(int8.some(isNegative));
// Expected output: true

console.log(positives.some(isNegative));
// Expected output: false
```

Syntax

```js
some(callbackFn)
some(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the typed array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the typed array.

`index`

The index of the current element being processed in the typed array.

`array`

The typed array `some()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`false` unless `callbackFn` returns a truthy value for a typed array element, in which case `true` is immediately returned.

Description

See `Array.prototype.some()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Testing size of all typed array elements

The following example tests whether any element in the typed array is bigger than 10.

```js
function isBiggerThan10(element, index, array) {
  return element > 10;
}
new Uint8Array([2, 5, 8, 1, 4]).some(isBiggerThan10); // false
new Uint8Array([12, 5, 8, 1, 4]).some(isBiggerThan10); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.some

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
some	45	12	37	32	10	45	37	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.some`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.some` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.every()`
- `TypedArray.prototype.forEach()`
- `TypedArray.prototype.find()`
- `TypedArray.prototype.includes()`
- `Array.prototype.some()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.sort()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/sort · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`sort()`** method of `TypedArray` instances sorts the elements of a typed array *in place* and returns the reference to the same typed array, now sorted. This method has the same algorithm as `Array.prototype.sort()`, except that it sorts the values numerically instead of as strings by default.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.sort()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([40, 10, 50, 20, 30]);
uint8.sort();

console.log(uint8);
// Expected output: Uint8Array [10, 20, 30, 40, 50]
```

Syntax

js

```
sort()
sort(compareFn)
```

Parameters

compareFn Optional

A function that determines the order of the elements. The function is called with the following arguments:

a

The first element for comparison.

b

The second element for comparison.

It should return a number where:

- A negative value indicates that `a` should come before `b`.
- A positive value indicates that `a` should come after `b`.
- Zero or `NaN` indicates that `a` and `b` are considered equal.

To memorize this, remember that `(a, b) => a - b` sorts numbers in ascending order.

If omitted, the typed array elements are sorted according to numeric value.

Return value

The reference to the original typed array, now sorted. Note that the typed array is sorted *in place*, and no copy is made.

Description

See `Array.prototype.sort()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using sort()

For more examples, see also the `Array.prototype.sort()` method.

js

```
let numbers = new Uint8Array([40, 1, 5, 200]);
numbers.sort();
// Uint8Array [ 1, 5, 40, 200 ]
// Unlike plain Arrays, a compare function is not required
// to sort the numbers numerically.

// Regular Arrays require a compare function to sort numerically:
numbers = [40, 1, 5, 200];
numbers.sort();
// [1, 200, 40, 5]

numbers.sort((a, b) => a - b); // compare numbers
// [ 1, 5, 40, 200 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.sort`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`sort`	45	12	46	32	10	45	46	32	10	5.0	45	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.sort`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.sort` with modern behavior like stable sort in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.reverse()`
- `TypedArray.prototype.toSorted()`
- `Array.prototype.sort()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.subarray()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/subarray · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`subarray()`** method of `TypedArray` instances returns a new typed array on the same `ArrayBuffer` store and with the same element types as for this typed array. The begin offset is **inclusive** and the end offset is **exclusive**.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.subarray()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

console.log(uint8.subarray(1, 3));
// Expected output: Uint8Array [20, 30]

console.log(uint8.subarray(1));
// Expected output: Uint8Array [20, 30, 40, 50]
```

Syntax

```js
subarray()
subarray(begin)
subarray(begin, end)
```

Parameters

`begin` Optional

Element to begin at. The offset is inclusive. The whole array will be included in the new view if this value is not specified.

`end` Optional

Element to end at. The offset is exclusive. If not specified, all elements from the one specified by `begin` to the end of the array are included in the new view.

Return value

A new `TypedArray` object.

Description

The range specified by `begin` and `end` is clamped to the valid index range for the current array; if the computed length of the new array would be negative, it's clamped to zero. If either `begin` or `end` is negative, it refers to an index from the end of the array instead of from the beginning.

Also note that this is creating a new view on the existing buffer; changes to the new object's contents will impact the original object and vice versa.

Examples

Using the subarray() method

```js
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);
uint8.set([1, 2, 3]);

console.log(uint8); // Uint8Array [ 1, 2, 3, 0, 0, 0, 0, 0 ]

const sub = uint8.subarray(0, 4);

console.log(sub); // Uint8Array [ 1, 2, 3, 0 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.subarray

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
subarray	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.subarray`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.subarray` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/toLocaleString · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since January 2017.

The **`toLocaleString()`** method of `TypedArray` instances returns a string representing the elements of the typed array. The elements are converted to strings using their `toLocaleString` methods and these strings are separated by a locale-specific string (such as a comma ","). This method has the same algorithm as `Array.prototype.toLocaleString()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.toLocaleString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint32Array([500, 8123, 12]);

console.log(uint8.toLocaleString());
// Expected output: "500,8123,12"

console.log(uint8.toLocaleString("en-GB"));
// Expected output: "500,8,123,12"

console.log(
  uint8.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// Expected output: "500,00 €,8.123,00 €,12,00 €"
```

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object with configuration properties. See `Number.prototype.toLocaleString()`.

Return value

A string representing the elements of the typed array.

Description

See `Array.prototype.toLocaleString()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using toLocaleString()

```js
const uint = new Uint32Array([2000, 500, 8123, 12, 4212]);

uint.toLocaleString();
// if run in a de-DE locale
// "2.000,500,8.123,12,4.212"

uint.toLocaleString("en-US");
// "2,000,500,8,123,12,4,212"

uint.toLocaleString("ja-JP", { style: "currency", currency: "JPY" });
// "￥2,000,￥500,￥8,123,￥12,￥4,212"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.tolocalestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	7	12	51	11.6	5.1	18	51	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.toString()`
- `Array.prototype.toLocaleString()`
- `Intl`
- `Intl.ListFormat`
- `Number.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.toReversed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/toReversed · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2023.

The **`toReversed()`** method of `TypedArray` instances is the copying counterpart of the `reverse()` method. It returns a new typed array with the elements in reversed order. This method has the same algorithm as `Array.prototype.toReversed()`.

Syntax

```
toReversed()
```

Parameters

None.

Return value

A new typed array containing the elements in reversed order.

Description

See `Array.prototype.toReversed()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using toReversed()

```js
const uint8 = new Uint8Array([1, 2, 3]);
const reversedUint8 = uint8.toReversed();
console.log(reversedUint8); // Uint8Array [3, 2, 1]
console.log(uint8); // Uint8Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.toreversed

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toReversed	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.toReversed`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.toReversed` in `core-js`
- JavaScript typed arrays guide
- `TypedArray.prototype.reverse()`
- `TypedArray.prototype.toSorted()`
- `TypedArray.prototype.with()`
- `Array.prototype.toReversed()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.toSorted()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/toSorted · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2023.

The **`toSorted()`** method of `TypedArray` instances is the copying version of the `sort()` method. It returns a new typed array with the elements sorted in ascending order. This method has the same algorithm as `Array.prototype.toSorted()`, except that it sorts the values numerically instead of as strings by default.

Syntax

```js
toSorted()
toSorted(compareFn)
```

Parameters

`compareFn` Optional

A function that determines the order of the elements. If omitted, the typed array elements are sorted according to numeric value. See `sort()` for more information.

Return value

A new typed array with the elements sorted in ascending order.

Description

See `Array.prototype.toSorted()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Sorting an array

For more examples, see also the `Array.prototype.sort()` method.

```js
const numbers = new Uint8Array([40, 1, 5, 200]);
const numberSorted = numbers.toSorted();
console.log(numberSorted); // Uint8Array [ 1, 5, 40, 200 ]
// Unlike plain Arrays, a compare function is not required
// to sort the numbers numerically.
console.log(numbers); // Uint8Array [ 40, 1, 5, 200 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.tosorted`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toSorted`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.toSorted`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.prototype.toSorted` in `core-js`
- JavaScript typed arrays guide
- `TypedArray.prototype.sort()`
- `TypedArray.prototype.toReversed()`
- `TypedArray.prototype.with()`
- `Array.prototype.toSorted()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/toString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since January 2017.

The **`toString()`** method of `TypedArray` instances returns a string representing the specified typed array and its elements. This method has the same algorithm as `Array.prototype.toString()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.toString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30, 40, 50]);

const uint8String = uint8.toString();

console.log(uint8String.startsWith("10"));
// Expected output: true
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

A string representing the elements of the typed array.

Description

See `Array.prototype.toString()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Converting a typed array to a string

```js
const uint8 = new Uint8Array([1, 2, 3]);
// Explicit conversion
console.log(uint8.toString()); // 1,2,3
// Implicit conversion
console.log(`${uint8}`); // 1,2,3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
toString	7	12	51	11.6	5.1	18	51	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.join()`
- `TypedArray.prototype.toLocaleString()`
- `Array.prototype.toString()`
- `String.prototype.toString()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.values()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/values · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`values()`** method of `TypedArray` instances returns a new *array iterator* object that iterates the value of each item in the typed array. This method has the same algorithm as `Array.prototype.values()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.values()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const bytes = new Uint8Array([10, 20, 30, 40, 50]);
const iterator = bytes.values();

iterator.next();
iterator.next();

console.log(iterator.next().value);
// Expected output: 30
```

Syntax

js

```
values()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

See `Array.prototype.values()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Iteration using for...of loop

js

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const values = arr.values();
for (const n of values) {
  console.log(n);
}
```

Alternative iteration

js

```
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const values = arr.values();
console.log(values.next().value); // 10
console.log(values.next().value); // 20
console.log(values.next().value); // 30
console.log(values.next().value); // 40
console.log(values.next().value); // 50
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.values`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`values`	38	12	37	25	10	38	37	25	10	3.0	38	10	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.values`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `TypedArray.prototype.values` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.entries()`
- `TypedArray.prototype.keys()`
- `TypedArray.prototype[Symbol.iterator]()`
- `Array.prototype.values()`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/with · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2023.

The **`with()`** method of `TypedArray` instances is the copying version of using the bracket notation to change the value of a given index. It returns a new typed array with the element at the given index replaced with the given value. This method has the same algorithm as `Array.prototype.with()`.

Syntax

```js
arrayInstance.with(index, value)
```

Parameters

`index`

Zero-based index at which to change the typed array, converted to an integer.

`value`

Any value to be assigned to the given index.

Return value

A new typed array with the element at `index` replaced with `value`.

Exceptions

`RangeError`

Thrown if `index >= array.length` or `index < -array.length`.

Description

See `Array.prototype.with()` for more details. This method is not generic and can only be called on typed array instances.

Examples

Using with()

```js
const arr = new Uint8Array([1, 2, 3, 4, 5]);
console.log(arr.with(2, 6)); // Uint8Array [1, 2, 6, 4, 5]
console.log(arr); // Uint8Array [1, 2, 3, 4, 5]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype.with`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.with`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.prototype.with` in `core-js`
- JavaScript typed arrays guide
- `TypedArray.prototype.toReversed()`
- `TypedArray.prototype.toSorted()`
- `TypedArray.prototype.at()`

- `Array.prototype.with()`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/Symbol.iterator · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`[Symbol.iterator]()`** method of `TypedArray` instances implements the iterable protocol and allows typed arrays to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns an array iterator object that yields the value of each index in the typed array.

The initial value of this property is the same function object as the initial value of the `TypedArray.prototype.values` property.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype[Symbol.iterator]()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const uint8 = new Uint8Array([10, 20, 30]);
const iterator = uint8[Symbol.iterator]();

for (const value of iterator) {
  console.log(value);
}

// Expected output: 10
// Expected output: 20
// Expected output: 30
```

Syntax

js

```
typedArray[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `TypedArray.prototype.values()`: a new iterable iterator object that yields the value of each index in the typed array.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes typed arrays iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```js
const arr = new Uint8Array([10, 20, 30, 40, 50]);
for (const n of arr) {
  console.log(n);
}
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```js
const arr = new Uint8Array([10, 20, 30, 40, 50]);
const arrIter = arr[Symbol.iterator]();
console.log(arrIter.next().value); // 10
console.log(arrIter.next().value); // 20
console.log(arrIter.next().value); // 30
console.log(arrIter.next().value); // 40
console.log(arrIter.next().value); // 50
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-%typedarray%.prototype-%symbol.iterator%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.iterator]	38	12	36[1,2,3,4]	25	10	38	36[1,2,3,4]	25	10	3.0	38	10	1.0.0	1.0	0.12.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.@@iterator`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `TypedArray.prototype[Symbol.iterator]` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `TypedArray.prototype.entries()`
- `TypedArray.prototype.keys()`
- `TypedArray.prototype.values()`
- `Symbol.iterator`

- Iteration protocols

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.buffer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/buffer · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`buffer`** accessor property of `TypedArray` instances returns the `ArrayBuffer` or `SharedArrayBuffer` referenced by this typed array at construction time.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.buffer". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint16 = new Uint16Array(buffer);

console.log(uint16.buffer.byteLength);
// Expected output: 8
```

Description

The `buffer` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the *TypedArray* is constructed and cannot be changed. *TypedArray* is one of the TypedArray objects.

Because a typed array is a *view* of a buffer, the underlying buffer may be longer than the typed array itself.

Examples

Using the buffer property

js

```
const buffer = new ArrayBuffer(8);
const uint16 = new Uint16Array(buffer);
uint16.buffer; // ArrayBuffer { byteLength: 8 }
```

Accessing the underlying buffer from a sliced array view

js

```
const buffer = new ArrayBuffer(1024);
const arr = new Uint8Array(buffer, 64, 128);
console.log(arr.byteLength); // 128
console.log(arr.buffer.byteLength); // 1024
console.log(arr.buffer === buffer); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-get-%typedarray%.prototype.buffer`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`buffer`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.buffer`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`

TypedArray.prototype.byteLength

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/byteLength · MDN page last modified 2025-09-25

Baseline: Widely available — available across major browsers since July 2015.

The **`byteLength`** accessor property of `TypedArray` instances returns the length (in bytes) of this typed array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.byteLength". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer, 2);

console.log(uint8.byteLength);
// Expected output: 6
```

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. If the typed array is length-tracking, then its length depends on the length of the underlying buffer, and may change if the buffer is resized. Otherwise, the value is established when the typed array is constructed and cannot be changed. Whether length-tracking or

not, the `byteLength` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteLength property

```js
const buffer = new ArrayBuffer(8);

const uint8 = new Uint8Array(buffer);
uint8.byteLength; // 8 (matches the byteLength of the buffer)

const uint8newLength = new Uint8Array(buffer, 1, 5);
uint8newLength.byteLength; // 5 (as specified when constructing the Uint8Array)

const uint8offset = new Uint8Array(buffer, 2);
uint8offset.byteLength; // 6 (due to the offset of the constructed Uint8Array)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const uint8lengthTracking = new Uint8Array(buffer2, 4);
uint8lengthTracking.byteLength; // 12 (16 - 4)
buffer2.resize(20);
uint8lengthTracking.byteLength; // 16 (20 - 4)
buffer2.resize(3);
uint8lengthTracking.byteLength; // 0 (viewed range is no longer valid)
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-get-%typedarray%.prototype.bytelength`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`byteLength`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.byteLength`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `TypedArray`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.byteOffset

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/byteOffset · MDN page last modified 2025-09-25

Baseline: Widely available — available across major browsers since July 2015.

The **`byteOffset`** accessor property of `TypedArray` instances returns the offset (in bytes) of this typed array from the start of its `ArrayBuffer` or `SharedArrayBuffer`.

Description

The `byteOffset` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the typed array is constructed and cannot be changed. However, the `byteOffset` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteOffset property

```js
const buffer = new ArrayBuffer(8);

const uint8array1 = new Uint8Array(buffer);
uint8array1.byteOffset; // 0 (no offset specified)

const uint8array2 = new Uint8Array(buffer, 3);
uint8array2.byteOffset; // 3 (as specified when constructing Uint8Array)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const uint8lengthTracking = new Uint8Array(buffer2, 4);
uint8lengthTracking.byteOffset; // 4
buffer2.resize(3);
uint8lengthTracking.byteOffset; // 0 (viewed range is no longer valid)
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-get-%typedarray%.prototype.byteoffset`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`byteOffset`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.byteOffset`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`

JavaScript › Reference › Standard built-in objects › TypedArray

TypedArray.prototype.length

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypedArray/length · MDN page last modified 2025-09-25

Baseline: Widely available — available across major browsers since July 2015.

The **`length`** accessor property of `TypedArray` instances returns the length (in elements) of this typed array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: TypedArray.prototype.length". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer, 2);

console.log(uint8.length);
// Expected output: 6
```

Description

The `length` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. If the typed array is length-tracking, then its length depends on the length of the underlying buffer, and may change if the buffer is resized. Otherwise, the value is established when the typed array is constructed and cannot be changed. Whether length-tracking or not, the `length` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the length property

js

```
const buffer = new ArrayBuffer(8);

const uint8 = new Uint8Array(buffer);
uint8.length; // 8 (matches the length of the buffer)

const uint8newLength = new Uint8Array(buffer, 1, 5);
uint8newLength.length; // 5 (as specified when constructing the Uint8Array)

const uint8offset = new Uint8Array(buffer, 2);
uint8offset.length; // 6 (due to the offset of the constructed Uint8Array)
```

(code continues on the next page)

(code continued from the previous page)

```
const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const uint8lengthTracking = new Uint8Array(buffer2, 4);
uint8lengthTracking.length; // 12 (16 - 4)
buffer2.resize(20);
uint8lengthTracking.length; // 16 (20 - 4)
buffer2.resize(3);
uint8lengthTracking.length; // 0 (viewed range is no longer valid)
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-get-%typedarray%.prototype.length`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`length`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypedArray.length`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`

JavaScript › Reference › Standard built-in objects

TypeError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypeError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`TypeError`** object represents an error when an operation could not be performed, typically (but not exclusively) when a value is not of the expected type.

A `TypeError` may be thrown when:

- an operand or argument passed to a function is incompatible with the type expected by that operator or function; or
- when attempting to modify a value that cannot be changed; or
- when attempting to use a value in an inappropriate way.

`TypeError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`TypeError` is a subclass of `Error`.

Constructor

`TypeError()`

Creates a new `TypeError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `TypeError.prototype` and shared by all `TypeError` instances.

`TypeError.prototype.constructor`

The constructor function that created the instance object. For `TypeError` instances, the initial value is the `TypeError` constructor.

`TypeError.prototype.name`

Represents the name for the type of error. For `TypeError.prototype.name`, the initial value is `"TypeError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a TypeError

```js
try {
  null.f();
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "null has no properties"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

Creating a TypeError

```js
try {
  throw new TypeError("Hello");
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-native-error-types-used-in-this-standard-typeerror

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
TypeError	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
TypeError() constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
TypeError is serializable	77	79	103[1,2,3]	64	**No**	77	103[1,2,3]	55	**No**	12.0	77	**No**	?	**No**	**No**

1. Version 103 serializable properties: `name`, `message`, `cause`, `fileName`, `lineNumber` and `columnNumber`.
2. Version 104 also serializes `stack` in the main thread (`window.postMessage()` and `structuredClone()`).
3. Version 110 also serializes `stack` in workers (`worker.postMessage()` and `structuredClone()`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypeError`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Error`

JavaScript › Reference › Standard built-in objects › TypeError

TypeError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/TypeError/TypeError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`TypeError()`** constructor creates `TypeError` objects.

Syntax

```js
new TypeError()
new TypeError(message)
new TypeError(message, options)
new TypeError(message, fileName)
new TypeError(message, fileName, lineNumber)

TypeError()
TypeError(message)
TypeError(message, options)
TypeError(message, fileName)
TypeError(message, fileName, lineNumber)
```

> **Note:** `TypeError()` can be called with or without `new`. Both create a new `TypeError` instance.

Parameters

`message` Optional

Human-readable description of the error

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The name of the file containing the code that caused the exception

`lineNumber` Optional Non-standard

The line number of the code that caused the exception

Examples

Catching a TypeError

```js
try {
  null.f();
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "null has no properties"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

Creating a TypeError

```js
try {
  throw new TypeError("Hello");
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "TypeError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-nativeerror-constructors`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`TypeError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.TypeError.TypeError`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`

JavaScript › Reference › Standard built-in objects

Uint8Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint8Array`** typed array represents an array of 8-bit unsigned integers. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint8Array` is a subclass of the hidden `TypedArray` class.

Description

`Uint8Array` is currently the only `TypedArray` subclass that has additional methods compared to other typed arrays. Because of its nature as a generic byte array, it is the most suitable for working with arbitrary binary data. It supports two sets of methods for the creation, serialization, and modification of `Uint8Array` data to/from hex strings and base64 strings.

- `Uint8Array.fromBase64()`, `Uint8Array.prototype.toBase64()`, and `Uint8Array.prototype.setFromBase64()` for working with base64 strings, where 3 bytes are encoded by 4 characters that are either 0–9, A–Z, a–z, "+", and "/" (or "-" and "_", if using URL-safe base64).
- `Uint8Array.fromHex()`, `Uint8Array.prototype.toHex()`, and `Uint8Array.prototype.setFromHex()` for working with hex strings, where every byte is encoded by two characters, each one being either 0–9 or A–F (case-insensitive).

Constructor

`Uint8Array()`

Creates a new `Uint8Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint8Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of `Uint8Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

`Uint8Array.fromBase64()`

Creates a new `Uint8Array` object from a base64-encoded string.

`Uint8Array.fromHex()`

Creates a new `Uint8Array` object from a hex-encoded string.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint8Array.prototype` and shared by all `Uint8Array` instances.

`Uint8Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of a `Uint8Array`.

`Uint8Array.prototype.constructor`

The constructor function that created the instance object. For `Uint8Array` instances, the initial value is the `Uint8Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

`Uint8Array.prototype.setFromBase64()`

Populates this `Uint8Array` object with bytes from a base64-encoded string, returning an object indicating how many bytes were read and written.

`Uint8Array.prototype.setFromHex()`

Populates this `Uint8Array` object with bytes from a hex-encoded string, returning an object indicating how many bytes were read and written.

`Uint8Array.prototype.toBase64()`

Returns a base64-encoded string based on the data in this `Uint8Array` object.

`Uint8Array.prototype.toHex()`

Returns a hex-encoded string based on the data in this `Uint8Array` object.

Examples

Different ways to create a Uint8Array

```js
// From a length
const uint8 = new Uint8Array(2);
uint8[0] = 42;
console.log(uint8[0]); // 42
console.log(uint8.length); // 2
console.log(uint8.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8FromIterable = new Uint8Array(iterable);
console.log(uint8FromIterable);
// Uint8Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`Uint8Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Uint8Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Uint8Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`fromBase64`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0
`fromHex`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0
`setFromBase64`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0
`setFromHex`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0
`toBase64`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0
`toHex`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Uint8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › Uint8Array

Uint8Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array/Uint8Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint8Array()`** constructor creates `Uint8Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Uint8Array()
new Uint8Array(length)
new Uint8Array(typedArray)
new Uint8Array(object)

new Uint8Array(buffer)
new Uint8Array(buffer, byteOffset)
new Uint8Array(buffer, byteOffset, length)
```

> **Note:** `Uint8Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint8Array

```js
// From a length
const uint8 = new Uint8Array(2);
uint8[0] = 42;
console.log(uint8[0]); // 42
console.log(uint8.length); // 2
console.log(uint8.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8FromIterable = new Uint8Array(iterable);
console.log(uint8FromIterable);
// Uint8Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Uint8Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Uint8Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array.Uint8Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › Uint8Array

Uint8Array.fromBase64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array/fromBase64 · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since September 2025; may not work in older ones.

The **`Uint8Array.fromBase64()`** static method creates a new `Uint8Array` object from a base64-encoded string.

This method should be preferred over `Window.atob()` because it results in a byte array, which is easier to work with than a string containing raw bytes, unless your decoded binary data is actually intended to be ASCII text. If you already have an array buffer allocated and you want to populate it, use the instance method `Uint8Array.prototype.setFromBase64()` instead.

Syntax

```js
Uint8Array.fromBase64(string)
Uint8Array.fromBase64(string, options)
```

Parameters

`string`

A base64 string encoding bytes to convert to a `Uint8Array`. The string must only contain characters in the base64 alphabet, which includes A–Z, a–z, 0–9, and two special characters, which are either + and / (if using `alphabet: "base64"` in `options`) or - and _ (if using `alphabet: "base64url"` in `options`). It may have padding = characters at the end. Any ASCII white space characters within the string are ignored.

`options` Optional

An object customizing the base64 string interpretation process. It can contain the following properties:

`alphabet` Optional

A string specifying the base64 alphabet to use. It can be one of the following:

`"base64"` (default)

Accept input encoded with the standard base64 alphabet, which uses + and `/`.

`"base64url"`

Accept input encoded with the URL-safe base64 alphabet, which uses - and _.

`lastChunkHandling` Optional

A string specifying how to handle the last chunk of the base64 string. Because every 4 characters in base64 encodes 3 bytes, the string is separated into chunks of 4 characters. If the last chunk has fewer than 4 characters, it needs to be handled differently. It can be one of the following:

`"loose"` (default)

The last chunk can either be 2 or 3 base64 characters, or exactly 4 characters long with padding = characters. The last chunk is decoded and appended to the result.

`"strict"`

The last chunk must be exactly 4 characters long with padding = characters. Furthermore, overflow bits (trailing bits from the last base64 character that don't represent any data) must be 0. The last chunk is decoded and appended to the result.

`"stop-before-partial"`

If the last chunk is exactly 4 characters long with padding = characters, then it's decoded and appended to the result. Otherwise, the last partial chunk is ignored (but if it contains one base64 character followed by =, then a syntax error is still thrown). This is useful if the string is coming from a stream and the last chunk is not yet complete. To know how many characters of the input were read, use `Uint8Array.prototype.setFromBase64()` instead (the linked page also contains an example of stream decoding using `"stop-before-partial"`).

Return value

A new `Uint8Array` object containing the decoded bytes from the base64-encoded string.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the specified alphabet, or if the last chunk does not satisfy the `lastChunkHandling` option.

`TypeError`

Thrown in one of the following cases:

- The input string is not a string.
- The `options` object is not an object or `undefined`.
- The options are not of the expected values or `undefined`.

Examples

Decoding a base64 string

This example uses the default `alphabet` and `lastChunkHandling` options to decode a base64 string. Note that:

- The whitespace in the space is ignored.
- The string has 14 base64 characters, not a multiple of 4. This is only valid and decodable with `lastChunkHandling: "loose"`.
- The last chunk, `Ph`, ends in the character `h` which is `0b100001` in base64, so the last `0001` bits are "overflow bits" and are ignored. This is only valid and decodable with `lastChunkHandling: "loose"`.

```js
const uint8Array = Uint8Array.fromBase64("PGI+ TURO PC9i Ph");
console.log(uint8Array); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]
```

Decoding a URL-safe base64 string

This example uses the `alphabet` option to decode a URL-safe base64 string.

```js
const uint8Array = Uint8Array.fromBase64("PGI-TUROPC9iPg", {
  alphabet: "base64url",
});
console.log(uint8Array); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]
```

Decoding a base64 string with strict last chunk handling

This example uses the `lastChunkHandling` option to decode a base64 string, where the last chunk must be exactly 4 characters long with padding `=` characters, and the overflow bits must be 0.

```js
const array1 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg==", {
  lastChunkHandling: "strict",
});
console.log(array1); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]

const array2 = Uint8Array.fromBase64("PGI+ TURO PC9i Ph==", {
  lastChunkHandling: "strict",
});
// Throws a SyntaxError because h is 0b100001, where the last 4 bits are not 0

const array3 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg", {
  lastChunkHandling: "strict",
});
// Throws a SyntaxError because the last chunk is not exactly 4 characters long
```

Decoding a base64 string with partial last chunk handling

This example uses the `lastChunkHandling` option to decode a base64 string, ignoring any partial last chunk.

```js
// The last chunk is complete
const array1 = Uint8Array.fromBase64("PGI+ TURO PC9i", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array1); // Uint8Array(9) [60, 98, 62, 77, 68, 78, 60, 47, 98]

// The last chunk is also complete with padding
const array2 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg==", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array2); // Uint8Array(10) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62]

// The last chunk is partial; it's ignored
const array3 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array3); // Uint8Array(9) [60, 98, 62, 77, 68, 78, 60, 47, 98]

// The last chunk is partial with padding; it's still ignored
const array4 = Uint8Array.fromBase64("PGI+ TURO PC9i Pg=", {
  lastChunkHandling: "stop-before-partial",
});
console.log(array4); // Uint8Array(9) [60, 98, 62, 77, 68, 78, 60, 47, 98]

// The last chunk is partial, but it contains one base64 character followed by `=`
const array5 = Uint8Array.fromBase64("PGI+ TURO PC9i P=", {
  lastChunkHandling: "stop-before-partial",
});
// Throws a SyntaxError because this cannot possibly be part of a valid base64 string
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-uint8array.frombase64

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fromBase64`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array.fromBase64`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint8Array.fromBase64` in `core-js`
- es-shims polyfill of `Uint8Array.fromBase64`
- `Uint8Array`
- `Uint8Array.prototype.setFromBase64()`
- `Uint8Array.prototype.toBase64()`
- `Window.atob()`

JavaScript › Reference › Standard built-in objects › Uint8Array

Uint8Array.fromHex()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array/fromHex · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since September 2025; may not work in older ones.

The **`Uint8Array.fromHex()`** static method creates a new `Uint8Array` object from a hexadecimal string.

This method parses the string into a byte array. To convert the string into a single number, use the `parseInt()` function with `radix` set to `16` instead.

Syntax

```js
Uint8Array.fromHex(string)
```

Parameters

`string`

A hexadecimal string encoding bytes to convert to a `Uint8Array`. The string must:

- Have an even number of characters because two characters encode one byte.
- Only contain characters in the hexadecimal alphabet, which includes 0–9 and A–F (case-insensitive).
- Not contain whitespace (unlike `Uint8Array.prototype.setFromBase64()`).

Return value

A new `Uint8Array` object containing the decoded bytes from the hexadecimal string.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the hex alphabet, or its length is odd.

`TypeError`

Thrown if the input string is not a string.

Examples

Decoding a hexadecimal string

This example decodes a hexadecimal string into a `Uint8Array`.

```js
const hexString = "cafed00d";
const bytes = Uint8Array.fromHex(hexString);
console.log(bytes); // Uint8Array [ 202, 254, 208, 13 ]
```

Uppercase characters are also supported:

```js
const hexString = "CAFEd00d";
const bytes = Uint8Array.fromHex(hexString);
console.log(bytes); // Uint8Array [ 202, 254, 208, 13 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-uint8array.fromhex

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fromHex`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array.fromHex`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint8Array.fromHex` in `core-js`
- es-shims polyfill of `Uint8Array.fromHex`
- `Uint8Array`
- `Uint8Array.prototype.setFromHex()`
- `Uint8Array.prototype.toHex()`
- `parseInt()`

JavaScript › Reference › Standard built-in objects › Uint8Array

Uint8Array.prototype.setFromBase64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array/setFromBase64 · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since September 2025; may not work in older ones.

The **`setFromBase64()`** method of `Uint8Array` instances populates this `Uint8Array` object with bytes from a base64-encoded string, returning an object indicating how many bytes were read and written.

This method is most suitable for populating a pre-allocated array buffer. If you just want to create a new `Uint8Array` object from a base64-encoded string, use the static method `Uint8Array.fromBase64()` instead.

Syntax

```js
setFromBase64(string)
setFromBase64(string, options)
```

Parameters

`string`

A base64 string encoding bytes to write into a `Uint8Array`. It has the same requirements as the `string` parameter of `Uint8Array.fromBase64()`. Note that the string is only read up to the point where the array is filled, so any invalid base64 syntax after that point is ignored.

`options` Optional

An object customizing the base64 string interpretation process. It has the same requirements as the `options` parameter of `Uint8Array.fromBase64()`.

Return value

An object containing the following properties:

`read`

The number of base64 characters read from the input string. If the decoded data fits into the array, this is the length of the input string (including padding); otherwise, it is the length up to the last complete 4-character chunk that fits into the array. Chunks will never be split (because the remaining bits cannot be partially "put back" into the base64 without completely re-encoding it); if the next chunk cannot fit into the remainder of the array, it will be entirely unread, resulting in the last one or two bytes of the array not being written.

`written`

The number of bytes written to the `Uint8Array`. Will never be greater than this `Uint8Array`'s `byteLength`.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the specified alphabet, or if the last chunk does not satisfy the `lastChunkHandling` option.

`TypeError`

Thrown in one of the following cases:

- The input string is not a string.
- The `options` object is not an object or `undefined`.
- The options are not of the expected values or `undefined`.

Examples

Decoding a base64 string

This example uses the default `alphabet` and `lastChunkHandling` options to decode a base64 string into an existing `Uint8Array`.

```js
const uint8Array = new Uint8Array(16);
const result = uint8Array.setFromBase64("PGI+ TURO PC9i Pg==");
console.log(result); // { read: 19, written: 10 }
console.log(uint8Array);
// Uint8Array(16) [60, 98, 62, 77, 68, 78, 60, 47, 98, 62, 0, 0, 0, 0, 0, 0]
```

Decoding a big string into a small array

If the string contains more data than the array can hold, the method will only write as many bytes as the array can hold, without discarding any bits.

```js
const uint8Array = new Uint8Array(8);
const result = uint8Array.setFromBase64("PGI+ TURO PC9i Pg==");
console.log(result); // { read: 9, written: 6 }
console.log(uint8Array);
// Uint8Array(8) [60, 98, 62, 77, 68, 78, 0, 0]
```

Note how the last two bytes of the array are not written. To decode these two bytes, we need to read at least three more base64 characters, which represent 18 bits. These can't fit into the remaining two bytes of the array, so we can only write 2 chunks, or 6 bytes.

Setting data at a specific offset

The `setFromBase64()` method always starts writing at the beginning of the `Uint8Array`. If you want to write to the middle of the array, you can write to a `TypedArray.prototype.subarray()` instead.

```js
const uint8Array = new Uint8Array(16);
// Start writing at offset 2
const result = uint8Array.subarray(2).setFromBase64("PGI+ TURO PC9i Pg==");
console.log(result); // { read: 19, written: 10 }
console.log(uint8Array);
// Uint8Array(16) [0, 0, 60, 98, 62, 77, 68, 78, 60, 47, 98, 62, 0, 0, 0, 0]
```

Stream decoding

This example is adapted from the original proposal. It mimics the `TextDecoder` API with the `stream` option. Note the use of `lastChunkHandling: "stop-before-partial"` to handle incomplete chunks.

```js
class Base64Decoder {
  #extra = "";

  decode(chunk = "", options = {}) {
    const opts = { ...options };
    // match TextEncoder API
    if (opts.stream) {
      opts.lastChunkHandling = "stop-before-partial";
    }
    chunk = this.#extra + chunk;
    this.#extra = "";
    // For simplicity, allocate new memory every time
    // the calculation below is guaranteed to be enough,
    // but may be too much if there is whitespace
    // if you're really concerned about memory, a TextDecoder style API is a bad choice
    let buffer = new Uint8Array(Math.ceil((chunk.length * 3) / 4));
    const { read, written } = buffer.setFromBase64(chunk, opts);
    buffer = buffer.subarray(0, written);
    this.#extra = chunk.slice(read);
    return buffer;
  }
}

const decoder = new Base64Decoder();

console.log(decoder.decode("SG Vsb ", { stream: true }));
// Uint8Array(3) [72, 101, 108]
console.log(decoder.decode("G8gV29ybGR ", { stream: true }));
// Uint8Array(6) [108, 111, 32, 87, 111, 114]
console.log(decoder.decode(""));
// Uint8Array(2) [108, 100]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-uint8array.prototype.setfrombase64`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setFromBase64`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array.setFromBase64`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Uint8Array.prototype.setFromBase64` in `core-js`
- es-shims polyfill of `Uint8Array.prototype.setFromBase64`
- `Uint8Array`
- `Uint8Array.fromBase64()`
- `Uint8Array.prototype.toBase64()`
- `Window.atob()`

JavaScript › Reference › Standard built-in objects › Uint8Array

Uint8Array.prototype.setFromHex()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array/setFromHex · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since September 2025; may not work in older ones.

The **`setFromHex()`** method of `Uint8Array` instances populates this `Uint8Array` object with bytes from a hex-encoded string, returning an object indicating how many bytes were read and written.

This method parses the string into a byte array. To convert the string into a single number, use the `parseInt()` function with `radix` set to `16` instead.

Syntax

```js
setFromHex(string)
```

Parameters

`string`

A hexadecimal string encoding bytes to write into a `Uint8Array`. The string must:

- Have an even number of characters because two characters encode one byte.
- Only contain characters in the hexadecimal alphabet, which includes 0–9 and A–F (case-insensitive).
- Not contain whitespace (unlike `Uint8Array.prototype.setFromBase64()`).

Note that the string is only read up to the point where the array is filled, so any invalid hex syntax after that point is ignored.

Return value

An object containing the following properties:

`read`

The number of hex characters read from the input string. If the decoded data fits into the array, it is the length of the input string: otherwise, it is the number of complete hex characters that fit into the array.

`written`

The number of bytes written to the `Uint8Array`. Will never be greater than this `Uint8Array`'s `byteLength`.

Exceptions

`SyntaxError`

Thrown if the input string contains characters outside the hex alphabet, or its length is odd.

`TypeError`

Thrown if the input string is not a string.

Examples

Decoding a hexadecimal string

This example decodes a hexadecimal string into an existing `Uint8Array`.

```js
const uint8Array = new Uint8Array(8);
const result = uint8Array.setFromHex("cafed00d");
console.log(result); // { read: 8, written: 4 }
console.log(uint8Array); // Uint8Array(8) [202, 254, 208, 13, 0, 0, 0, 0]
```

Decoding a big string into a small array

If the string contains more data than the array can hold, the method will only write as many bytes as the array can hold.

```js
const uint8Array = new Uint8Array(4);
const result = uint8Array.setFromHex("cafed00d-some random stuff");
console.log(result); // { read: 8, written: 4 }
console.log(uint8Array); // Uint8Array(4) [202, 254, 208, 13]
```

Excess characters are ignored, even if they are invalid. However the total length of the input string must be even.

Setting data at a specific offset

The `setFromHex()` method always starts writing at the beginning of the `Uint8Array`. If you want to write to the middle of the array, you can write to a `TypedArray.prototype.subarray()` instead.

```js
const uint8Array = new Uint8Array(8);
// Start writing at offset 2
const result = uint8Array.subarray(2).setFromHex("cafed00d");
console.log(result); // { read: 8, written: 4 }
console.log(uint8Array);
// Uint8Array(8) [0, 0, 202, 254, 208, 13, 0, 0]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-uint8array.prototype.setfromhex`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setFromHex`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array.setFromHex`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint8Array.prototype.setFromHex` in `core-js`
- es-shims polyfill of `Uint8Array.prototype.setFromHex`
- `Uint8Array`
- `Uint8Array.fromHex()`
- `Uint8Array.prototype.toHex()`
- `parseInt()`

JavaScript › Reference › Standard built-in objects › Uint8Array

Uint8Array.prototype.toBase64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array/toBase64 · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since September 2025; may not work in older ones.

The **`toBase64()`** method of `Uint8Array` instances returns a base64-encoded string based on the data in this `Uint8Array` object.

This method should be preferred over `Window.btoa()`, especially if you already have a `Uint8Array` holding the object, because you don't need to convert it to a string first.

Syntax

```js
toBase64()
toBase64(options)
```

Parameters

`options` Optional

An object customizing the base64 string format. It can contain the following properties:

`alphabet` Optional

A string specifying the base64 alphabet to use. It can be one of the following:

`"base64"` (default)

Encode input with the standard base64 alphabet, which uses `+` and `/`.

`"base64url"`

Encode input with the URL-safe base64 alphabet, which uses `-` and `_`.

`omitPadding` Optional

A boolean specifying whether to omit padding characters (`=`) at the end of the base64 string. The default is `false`.

Return value

A base64-encoded string representing the data in the `Uint8Array`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- The `options` object is not an object or `undefined`.
- The `options.alphabet` is not of the expected values or `undefined`.

Examples

Encoding binary data

This example uses the default `alphabet` and `omitPadding` options to encode data from a `Uint8Array` into a base64 string.

```js
const uint8Array = new Uint8Array([29, 233, 101, 161]);
console.log(uint8Array.toBase64()); // "HelloQ=="
```

Encoding data without padding

```js
const uint8Array = new Uint8Array([29, 233, 101, 161]);
console.log(uint8Array.toBase64({ omitPadding: true })); // "HelloQ"
```

Encoding data with URL-safe alphabet

This example populates a `URLSearchParams` object with a base64-encoded string using the URL-safe alphabet.

```js
const uint8Array = new Uint8Array([46, 139, 222, 255, 42, 46]);
const base64 = uint8Array.toBase64({ alphabet: "base64url" });
const params = new URLSearchParams();
params.set("data", base64);
console.log(params.toString()); // "data=Love_you"
```

Stream encoding

This example is adapted from the original proposal, showcasing how to implement streaming in userland. It mimics the `TextEncoder` API with the `stream` option.

js

```
class Base64Encoder {
  #extra;
  #extraLength;
  constructor() {
    this.#extra = new Uint8Array(3);
    this.#extraLength = 0;
  }

  // Partly derived from
https://github.com/lucacasonato/base64_streams/blob/main/src/iterator/encoder.ts
  encode(chunk = Uint8Array.of(), options = {}) {
    const stream = options.stream ?? false;

    if (this.#extraLength > 0) {
      const bytesNeeded = 3 - this.#extraLength;
      const bytesAvailable = Math.min(bytesNeeded, chunk.length);
      this.#extra.set(chunk.subarray(0, bytesAvailable), this.#extraLength);
      chunk = chunk.subarray(bytesAvailable);
      this.#extraLength += bytesAvailable;
    }

    if (!stream) {
      // assert: this.#extraLength.length === 0 || this.#extraLength === 3 || chunk.length
=== 0
      const prefix = this.#extra.subarray(0, this.#extraLength).toBase64();
      this.#extraLength = 0;
      return prefix + chunk.toBase64();
    }

    let extraReturn = "";

    if (this.#extraLength === 3) {
      extraReturn = this.#extra.toBase64();
      this.#extraLength = 0;
    }

    const remainder = chunk.length % 3;
    if (remainder > 0) {
      this.#extra.set(chunk.subarray(chunk.length - remainder));
      this.#extraLength = remainder;
      chunk = chunk.subarray(0, chunk.length - remainder);
    }

    return extraReturn + chunk.toBase64();
  }
}

const encoder = new Base64Encoder();

console.log(
  encoder.encode(Uint8Array.of(72, 101, 108, 108, 111), { stream: true }),
);
// "SGVs"
console.log(
  encoder.encode(Uint8Array.of(32, 87, 111, 114, 108, 100), { stream: true }),
);
// "bG8gV29y"
console.log(encoder.encode());
// "bGQ="
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-uint8array.prototype.tobase64`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toBase64`	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array.toBase64`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint8Array.prototype.toBase64` in `core-js`
- es-shims polyfill of `Uint8Array.prototype.toBase64`
- `Uint8Array`
- `Uint8Array.fromBase64()`
- `Uint8Array.prototype.setFromBase64()`
- `Window.btoa()`

JavaScript › Reference › Standard built-in objects › Uint8Array

Uint8Array.prototype.toHex()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8Array/toHex · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since September 2025; may not work in older ones.

The **`toHex()`** method of `Uint8Array` instances returns a hex-encoded string based on the data in this `Uint8Array` object.

This method creates strings from a byte array. To convert individual numbers into hex, use the `Number.prototype.toString()` method with `radix` set to `16` instead.

Syntax

```js
toHex()
```

Parameters

None.

Return value

A hex-encoded string representing the data in the `Uint8Array`.

Examples

Encoding binary data

This example encodes data from a `Uint8Array` into a hex string.

```js
const uint8Array = new Uint8Array([202, 254, 208, 13]);
console.log(uint8Array.toHex()); // "cafed00d"

const data = new Uint8Array([255, 0, 0, 0, 255, 0, 0, 0, 255]);
for (let i = 0; i < data.length; i += 3) {
  console.log(data.slice(i, i + 3).toHex());
}
// "ff0000"
// "00ff00"
// "0000ff"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-uint8array.prototype.tohex

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toHex	140	140	133	124	18.2	140	133	92	18.2	30.0	140	18.2	1.1.22	2.5.0	25.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8Array.toHex`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Uint8Array.prototype.toHex` in `core-js`
- es-shims polyfill of `Uint8Array.prototype.toHex`
- `Uint8Array`
- `Uint8Array.fromHex()`
- `Uint8Array.prototype.setFromHex()`
- `Number.prototype.toString()`

JavaScript › Reference › Standard built-in objects

Uint8ClampedArray

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8ClampedArray · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint8ClampedArray`** typed array represents an array of 8-bit unsigned integers clamped to 0–255. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint8ClampedArray` is a subclass of the hidden `TypedArray` class.

Constructor

`Uint8ClampedArray()`

Creates a new `Uint8ClampedArray` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint8ClampedArray.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of `Uint8ClampedArray`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint8ClampedArray.prototype` and shared by all `Uint8ClampedArray` instances.

`Uint8ClampedArray.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of a `Uint8ClampedArray`.

`Uint8ClampedArray.prototype.constructor`

The constructor function that created the instance object. For `Uint8ClampedArray` instances, the initial value is the `Uint8ClampedArray` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Uint8ClampedArray

```js
// From a length
const uint8c = new Uint8ClampedArray(2);
uint8c[0] = 42;
uint8c[1] = 1337;
console.log(uint8c[0]); // 42
console.log(uint8c[1]); // 255 (clamped)
console.log(uint8c.length); // 2
console.log(uint8c.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8ClampedArray([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8ClampedArray(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8ClampedArray(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8cFromIterable = new Uint8ClampedArray(iterable);
console.log(uint8cFromIterable);
// Uint8ClampedArray [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Uint8ClampedArray	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Uint8ClampedArray() constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
new Uint8ClampedArray(iterable)	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8ClampedArray`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint8ClampedArray` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › Uint8ClampedArray

Uint8ClampedArray() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint8ClampedArray/Uint8ClampedArray · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint8ClampedArray()`** constructor creates `Uint8ClampedArray` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Uint8ClampedArray()
new Uint8ClampedArray(length)
new Uint8ClampedArray(typedArray)
new Uint8ClampedArray(object)

new Uint8ClampedArray(buffer)
new Uint8ClampedArray(buffer, byteOffset)
new Uint8ClampedArray(buffer, byteOffset, length)
```

> **Note:** `Uint8ClampedArray()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint8ClampedArray

```js
// From a length
const uint8c = new Uint8ClampedArray(2);
uint8c[0] = 42;
uint8c[1] = 1337;
console.log(uint8c[0]); // 42
console.log(uint8c[1]); // 255 (clamped)
console.log(uint8c.length); // 2
console.log(uint8c.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Uint8ClampedArray([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint8ClampedArray(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Uint8ClampedArray(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint8cFromIterable = new Uint8ClampedArray(iterable);
console.log(uint8cFromIterable);
// Uint8ClampedArray [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Uint8ClampedArray()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Uint8ClampedArray(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint8ClampedArray.Uint8ClampedArray`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint8ClampedArray` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Uint16Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint16Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint16Array`** typed array represents an array of 16-bit unsigned integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint16Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Uint16Array()`

Creates a new `Uint16Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint16Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of `Uint16Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint16Array.prototype` and shared by all `Uint16Array` instances.

`Uint16Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of a `Uint16Array`.

`Uint16Array.prototype.constructor`

The constructor function that created the instance object. For `Uint16Array` instances, the initial value is the `Uint16Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Uint16Array

```js
// From a length
const uint16 = new Uint16Array(2);
uint16[0] = 42;
console.log(uint16[0]); // 42
console.log(uint16.length); // 2
console.log(uint16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Uint16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Uint16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint16FromIterable = new Uint16Array(iterable);
console.log(uint16FromIterable);
// Uint16Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Uint16Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Uint16Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Uint16Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint16Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint16Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › Uint16Array

Uint16Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint16Array/Uint16Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint16Array()`** constructor creates `Uint16Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Uint16Array()
new Uint16Array(length)
new Uint16Array(typedArray)
new Uint16Array(object)

new Uint16Array(buffer)
new Uint16Array(buffer, byteOffset)
new Uint16Array(buffer, byteOffset, length)
```

> **Note:** `Uint16Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint16Array

```js
// From a length
const uint16 = new Uint16Array(2);
uint16[0] = 42;
console.log(uint16[0]); // 42
console.log(uint16.length); // 2
console.log(uint16.BYTES_PER_ELEMENT); // 2
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Uint16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Uint16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint16FromIterable = new Uint16Array(iterable);
console.log(uint16FromIterable);
// Uint16Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Uint16Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Uint16Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint16Array.Uint16Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint16Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Uint32Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint32Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint32Array`** typed array represents an array of 32-bit unsigned integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Uint32Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Uint32Array()`

Creates a new `Uint32Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Uint32Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of `Uint32Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Uint32Array.prototype` and shared by all `Uint32Array` instances.

`Uint32Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of a `Uint32Array`.

`Uint32Array.prototype.constructor`

The constructor function that created the instance object. For `Uint32Array` instances, the initial value is the `Uint32Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Uint32Array

```js
// From a length
const uint32 = new Uint32Array(2);
uint32[0] = 42;
console.log(uint32[0]); // 42
console.log(uint32.length); // 2
console.log(uint32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Uint32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Uint32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint32FromIterable = new Uint32Array(iterable);
console.log(uint32FromIterable);
// Uint32Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Uint32Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Uint32Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Uint32Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint32Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › Uint32Array

Uint32Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Uint32Array/Uint32Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Uint32Array()`** constructor creates `Uint32Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Uint32Array()
new Uint32Array(length)
new Uint32Array(typedArray)
new Uint32Array(object)

new Uint32Array(buffer)
new Uint32Array(buffer, byteOffset)
new Uint32Array(buffer, byteOffset, length)
```

> **Note:** `Uint32Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Uint32Array

```js
// From a length
const uint32 = new Uint32Array(2);
uint32[0] = 42;
console.log(uint32[0]); // 42
console.log(uint32.length); // 2
console.log(uint32.BYTES_PER_ELEMENT); // 4
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Uint32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Uint32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Uint32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const uint32FromIterable = new Uint32Array(iterable);
console.log(uint32FromIterable);
// Uint32Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Uint32Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Uint32Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Uint32Array.Uint32Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Uint32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

undefined

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/undefined · MDN page last modified 2026-01-10

Baseline: Widely available — available across major browsers since July 2015.

The **`undefined`** global property represents the primitive value `undefined`. It is one of JavaScript's primitive types.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: undefined". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function test(t) {
  if (t === undefined) {
    return "Undefined value!";
  }
  return t;
}

let x;

console.log(test(x));
// Expected output: "Undefined value!"
```

Value

The primitive value `undefined`.

Property attributes of `undefined`	
Writable	no
Enumerable	no
Configurable	no

Description

`undefined` is a property of the *global object*. That is, it is a variable in global scope.

In all non-legacy browsers, `undefined` is a non-configurable, non-writable property. Even when this is not the case, avoid overriding it.

A variable that has not been assigned a value is of type `undefined`. A function returns `undefined` if a value was not `returned`. Accessing a property that does not exist also returns `undefined`. The `void` operator always returns `undefined`.

Note: While you can use `undefined` as an identifier (variable name) in any scope other than the global scope (because `undefined` is not a reserved word), doing so is a very bad idea that will make your code difficult to maintain and debug.

js · ✗ bad example

```
// DON'T DO THIS

(() => {
  const undefined = "foo";
  console.log(undefined, typeof undefined); // foo string
})();

((undefined) => {
  console.log(undefined, typeof undefined); // foo string
})("foo");
```

Examples

Strict equality and undefined

You can use `undefined` and the strict equality and inequality operators to determine whether a variable has a value. In the following code, the variable `x` is not initialized, and the `if` statement evaluates to true.

js

```
let x;
if (x === undefined) {
  // these statements execute
} else {
  // these statements do not execute
}
```

Note: The *strict equality* operator (as opposed to the *loose equality* operator) must be used here, because `x == undefined` also checks whether `x` is `null`, while strict equality doesn't. This is because `null` is not equivalent to `undefined`.

See Equality comparison and sameness for details.

typeof operator and undefined

`typeof` can also determine whether a variable is `undefined`:

js

```
let x;
if (typeof x === "undefined") {
  // these statements execute
}
```

One reason to use `typeof` is that it does not throw an error if the variable does not exist in the current scope.

```js
// x has not been declared before
// evaluates to true without errors
if (typeof x === "undefined") {
  // these statements execute
}

// Throws a ReferenceError
if (x === undefined) {
}
```

It also works with variables declared with `var` *after* the check, because the declaration is hoisted to the top of the scope with value `undefined`.

```js
if (typeof x === "undefined") {
  // these statements execute
}
var x = 1;
```

This technique is usually only useful for testing global variables. You can know if a variable exists at any other scope (blocks, functions, modules, etc.) just by looking at the source code. The global scope is bound to the global object, so checking the existence of a variable in the global context can be done by checking the existence of a property on the *global object*, such as by using the `in` operator:

```js
if ("x" in window) {
  // These statements execute only if x is defined globally
}
```

However, none of the techniques above work if the variable is declared with `let`, `const`, or other lexical declarations. Using `typeof` before the line of declaration still produces a `ReferenceError`, due to the temporal dead zone (TDZ).

js · ✗ bad example

```js
if (typeof z === "undefined") {
  // Uncaught ReferenceError: Cannot access 'z' before initialization
}
let z = 1;
```

Furthermore, `let` and `const` declarations do not create properties on the global object, so they cannot be checked with the `in` operator either.

js · ✗ bad example

```js
let z;
if ("z" in window) {
  // false, even if z is declared globally with let or const
}
```

If you want to share global variables across different scripts, it is more advisable to use `var` or explicitly attach them to the global object:

```js
window.myGlobalVar = "foo";
```

void operator and undefined

The `void` operator can also be used to produce the `undefined` value. This is very commonly seen in minified code because `void 0` is 3 bytes shorter and cannot be overridden. You should usually avoid this pattern in your own code.

```js
let x;
if (x === void 0) {
  // these statements execute
}

// y has not been declared before
if (y === void 0) {
  // throws Uncaught ReferenceError: y is not defined
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/global-object.html#sec-undefined`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`undefined`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.undefined`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript data types and data structures
- `null`

JavaScript › Reference › Standard built-in objects

unescape()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/unescape · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** `unescape()` is a non-standard function implemented by browsers and was only standardized for cross-engine compatibility. It is not required to be implemented by all JavaScript engines and may not work everywhere. Use `decodeURIComponent()` or `decodeURI()` if possible.

The **`unescape()`** function computes a new string in which hexadecimal escape sequences are replaced with the characters that they represent. The escape sequences might be introduced by a function like `escape()`.

Syntax

```js
unescape(str)
```

Parameters

`str`

A string to be decoded.

Return value

A new string in which certain characters have been unescaped.

Description

`unescape()` is a function property of the global object.

The `unescape()` function replaces any escape sequence with the character that it represents. Specifically, it replaces any escape sequence of the form `%XX` or `%uXXXX` (where `X` represents one hexadecimal digit) with the character that has the hexadecimal value `XX`/`XXXX`. If the escape sequence is not a valid escape sequence (for example, if `%` is followed by one or no hex digit), it is left as-is.

> **Note:** This function was used mostly for percent-encoding and is partly based on the escape format in RFC 1738. The `unescape()` function does *not* evaluate escape sequences in string literals. You can replace `\xXX` with `%XX` and `\uXXXX` with `%uXXXX` to get a string that can be handled by `unescape()`.

Examples

Using unescape()

```js
unescape("abc123"); // "abc123"
unescape("%E4%F6%FC"); // "äöü"
unescape("%u0107"); // "ć"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-unescape-string`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`unescape` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.unescape`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `unescape` in `core-js`
- `decodeURI`
- `decodeURIComponent`
- `escape`

JavaScript › Reference › Standard built-in objects

URIError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/URIError · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`URIError`** object represents an error when a global URI handling function was used in a wrong way.

`URIError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`URIError` is a subclass of `Error`.

Constructor

`URIError()`

Creates a new `URIError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `URIError.prototype` and shared by all `URIError` instances.

`URIError.prototype.constructor`

The constructor function that created the instance object. For `URIError` instances, the initial value is the `URIError` constructor.

`URIError.prototype.name`

Represents the name for the type of error. For `URIError.prototype.name`, the initial value is `"URIError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching a URIError

```js
try {
  decodeURIComponent("%");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "malformed URI sequence"
  console.log(e.name); // "URIError"
  console.log(e.stack); // Stack of the error
}
```

Creating a URIError

```js
try {
  throw new URIError("Hello");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "URIError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-native-error-types-used-in-this-standard-urierror

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
URIError	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
URIError() constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
URIError is serializable	77	79	103[1,2,3]	64	**No**	77	103[1,2,3]	55	**No**	12.0	77	**No**	?	**No**	**No**

1. Version 103 serializable properties: name, message, cause, fileName, lineNumber and columnNumber.
2. Version 104 also serializes stack in the main thread (window.postMessage() and structuredClone()).
3. Version 110 also serializes stack in workers (worker.postMessage() and structuredClone()).

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.URIError. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Error
- decodeURI()
- decodeURIComponent()
- encodeURI()
- encodeURIComponent()

JavaScript › Reference › Standard built-in objects › URIError

URIError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/URIError/URIError · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`URIError()`** constructor creates `URIError` objects.

Syntax

```js
new URIError()
new URIError(message)
new URIError(message, options)
new URIError(message, fileName)
new URIError(message, fileName, lineNumber)

URIError()
URIError(message)
URIError(message, options)
URIError(message, fileName)
URIError(message, fileName, lineNumber)
```

> **Note:** `URIError()` can be called with or without `new`. Both create a new `URIError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The name of the file containing the code that caused the exception.

`lineNumber` Optional Non-standard

The line number of the code that caused the exception.

Examples

Catching a URIError

```js
try {
  decodeURIComponent("%");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "malformed URI sequence"
  console.log(e.name); // "URIError"
  console.log(e.stack); // Stack of the error
}
```

Creating a URIError

```js
try {
  throw new URIError("Hello");
} catch (e) {
  console.log(e instanceof URIError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "URIError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-nativeerror-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`URIError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.URIError.URIError`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`

JavaScript › Reference › Standard built-in objects

WeakMap

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

A **`WeakMap`** is a collection of key/value pairs whose keys must be objects or non-registered symbols, with values of any arbitrary JavaScript type, and which does not create strong references to its keys. That is, an object's presence as a key in a `WeakMap` does not prevent the object from being garbage collected. Once an object used as a key has been collected, its corresponding values in any `WeakMap` become candidates for garbage collection as well — as long as they aren't strongly referred to elsewhere. The only primitive type that can be used as a `WeakMap` key is symbol — more specifically, non-registered symbols — because non-registered symbols are guaranteed to be unique and cannot be re-created.

`WeakMap` allows associating data to objects in a way that doesn't prevent the key objects from being collected, even if the values reference the keys. However, a `WeakMap` doesn't allow observing the liveness of its keys, which is why it doesn't allow enumeration; if a `WeakMap` exposed any method to obtain a list of its keys, the list would depend on the state of garbage collection, introducing non-determinism. If you want to have a list of keys, you should use a `Map` rather than a `WeakMap`.

You can learn more about `WeakMap` in the WeakMap object section of the Keyed collections guide.

Description

Keys of WeakMaps must be garbage-collectable. Most primitive data types can be arbitrarily created and don't have a lifetime, so they cannot be used as keys. Objects and non-registered symbols can be used as keys because they are garbage-collectable.

Key equality

Like regular `Map`, value equality is based on the SameValueZero algorithm, which is the same as the `===` operator because `WeakMap` can only hold object and symbol keys. This means that for object keys, equality is based on object identity. They are compared by reference, not by value.

Why WeakMap?

A map API *could* be implemented in JavaScript with two arrays (one for keys, one for values) shared by the four API methods. Setting elements on this map would involve pushing a key and value onto the end of each of those arrays simultaneously. As a result, the indices of the key and value would correspond to both arrays. Getting values from the map would involve iterating through all keys to find a match, then using the index of this match to retrieve the corresponding value from the array of values.

Such an implementation would have two main inconveniences:

1. The first one is an `O(n)` set and search (*n* being the number of keys in the map) since both operations must iterate through the list of keys to find a matching value.
2. The second inconvenience is a memory leak because the arrays ensure that references to each key and each value are maintained indefinitely. These references prevent the keys from being garbage collected, even if there are no other references to the object. This would also prevent the corresponding values from being garbage collected.

By contrast, in a `WeakMap`, a key object refers strongly to its contents as long as the key is not garbage collected, but weakly from then on. As such, a `WeakMap`:

- does not prevent garbage collection, which eventually removes references to the key object

- allows garbage collection of any values if their key objects are not referenced from somewhere other than a `WeakMap`

A `WeakMap` can be a particularly useful construct when mapping keys to information about the key that is valuable *only if* the key has not been garbage collected.

But because a `WeakMap` doesn't allow observing the liveness of its keys, its keys are not enumerable. There is no method to obtain a list of the keys. If there were, the list would depend on the state of garbage collection, introducing non-determinism. If you want to have a list of keys, you should use a `Map`.

Constructor

`WeakMap()`
Creates a new `WeakMap` object.

Instance properties

These properties are defined on `WeakMap.prototype` and shared by all `WeakMap` instances.

`WeakMap.prototype.constructor`
The constructor function that created the instance object. For `WeakMap` instances, the initial value is the `WeakMap` constructor.

`WeakMap.prototype[Symbol.toStringTag]`
The initial value of the `[Symbol.toStringTag]` property is the string `"WeakMap"`. This property is used in `Object.prototype.toString()`.

Instance methods

`WeakMap.prototype.delete()`
Removes the entry specified by the key from this `WeakMap`.

`WeakMap.prototype.get()`
Returns the value corresponding to the key in this `WeakMap`, or `undefined` if there is none.

`WeakMap.prototype.getOrInsert()`
Returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

`WeakMap.prototype.getOrInsertComputed()`
Returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

`WeakMap.prototype.has()`
Returns a boolean indicating whether an entry with the specified key exists in this `WeakMap` or not.

`WeakMap.prototype.set()`
Adds a new entry with a specified key and value to this `WeakMap`, or updates an existing entry if the key already exists.

Examples

Using WeakMap

```js
const wm1 = new WeakMap();
const wm2 = new WeakMap();
const wm3 = new WeakMap();
const o1 = {};
const o2 = () => {};
const o3 = window;

wm1.set(o1, 37);
wm1.set(o2, "azerty");
wm2.set(o1, o2); // a value can be anything, including an object or a function
wm2.set(o2, undefined);
wm2.set(wm1, wm2); // keys and values can be any objects. Even WeakMaps!

wm1.get(o2); // "azerty"
wm2.get(o2); // undefined, because that is the set value
wm2.get(o3); // undefined, because there is no key for o3 on wm2

wm1.has(o2); // true
wm2.has(o2); // true (even if the value itself is 'undefined')
wm2.has(o3); // false

wm3.set(o1, 37);
wm3.get(o1); // 37

wm1.has(o1); // true
wm1.delete(o1);
wm1.has(o1); // false
```

Implementing a WeakMap-like class with a .clear() method

```js
class ClearableWeakMap {
  #wm;
  constructor(init) {
    this.#wm = new WeakMap(init);
  }
  clear() {
    this.#wm = new WeakMap();
  }
  delete(k) {
    return this.#wm.delete(k);
  }
  get(k) {
    return this.#wm.get(k);
  }
  has(k) {
    return this.#wm.has(k);
  }
  set(k, v) {
    this.#wm.set(k, v);
    return this;
  }
}
```

Emulating private members

Developers can use a `WeakMap` to associate private data to an object, with the following benefits:

- Compared to a `Map`, a WeakMap does not hold strong references to the object used as the key, so the metadata shares the same lifetime as the object itself, avoiding memory leaks.
- Compared to using non-enumerable and/or `Symbol` properties, a WeakMap is external to the object and there is no way for user code to retrieve the metadata through reflective methods like `Object.getOwnPropertySymbols`.
- Compared to a closure, the same WeakMap can be reused for all instances created from a constructor, making it more memory-efficient, and allows different instances of the same class to read the private members of each other.

```js
let Thing;

{
  const privateScope = new WeakMap();
  let counter = 0;

  Thing = function () {
    this.someProperty = "foo";

    privateScope.set(this, {
      hidden: ++counter,
    });
  };

  Thing.prototype.showPublic = function () {
    return this.someProperty;
  };

  Thing.prototype.showPrivate = function () {
    return privateScope.get(this).hidden;
  };
}

console.log(typeof privateScope);
// "undefined"

const thing = new Thing();

console.log(thing);
// Thing {someProperty: "foo"}

thing.showPublic();
// "foo"

thing.showPrivate();
// 1
```

This is roughly equivalent to the following, using private fields:

```js
class Thing {
  static #counter = 0;
  #hidden;
  constructor() {
```

(code continues on the next page)

(code continued from the previous page)

```
    this.someProperty = "foo";
    this.#hidden = ++Thing.#counter;
  }
  showPublic() {
    return this.someProperty;
  }
  showPrivate() {
    return this.#hidden;
  }
}

const thing = new Thing();

console.log(thing);
// Thing {someProperty: "foo"}

thing.showPublic();
// "foo"

thing.showPrivate();
// 1
```

Associating metadata

A `WeakMap` can be used to associate metadata with an object, without affecting the lifetime of the object itself. This is very similar to the private members example, since private members are also modeled as external metadata that doesn't participate in prototypical inheritance.

This use case can be extended to already-created objects. For example, on the web, we may want to associate extra data with a DOM element, which the DOM element may access later. A common approach is to attach the data as a property:

js

```
const buttons = document.querySelectorAll(".button");
buttons.forEach((button) => {
  button.clicked = false;
  button.addEventListener("click", () => {
    button.clicked = true;
    const currentButtons = [...document.querySelectorAll(".button")];
    if (currentButtons.every((button) => button.clicked)) {
      console.log("All buttons have been clicked!");
    }
  });
});
```

This approach works, but it has a few pitfalls:

- The `clicked` property is enumerable, so it will show up in `Object.keys(button)`, `for...in` loops, etc. This can be mitigated by using `Object.defineProperty()`, but that makes the code more verbose.
- The `clicked` property is a normal string property, so it can be accessed and overwritten by other code. This can be mitigated by using a `Symbol` key, but the key would still be accessible via `Object.getOwnPropertySymbols()`.

Using a `WeakMap` fixes these:

```js
const buttons = document.querySelectorAll(".button");
const clicked = new WeakMap();
buttons.forEach((button) => {
  clicked.set(button, false);
  button.addEventListener("click", () => {
    clicked.set(button, true);
    const currentButtons = [...document.querySelectorAll(".button")];
    if (currentButtons.every((button) => clicked.get(button))) {
      console.log("All buttons have been clicked!");
    }
  });
});
```

Here, only code that has access to `clicked` knows the clicked state of each button, and external code can't modify the states. In addition, if any of the buttons gets removed from the DOM, the associated metadata will automatically get garbage-collected.

Caching

You can associate objects passed to a function with the result of the function, so that if the same object is passed again, the cached result can be returned without re-executing the function. This is useful if the function is pure (i.e., it doesn't mutate any outside objects or cause other observable side effects).

```js
const cache = new WeakMap();
function handleObjectValues(obj) {
  if (cache.has(obj)) {
    return cache.get(obj);
  }
  const result = Object.values(obj).map(heavyComputation);
  cache.set(obj, result);
  return result;
}
```

This only works if your function's input is an object. Moreover, even if the input is never passed in again, the result still remains forever in the cache as long as the key (input) is alive. A more effective way is to use a `Map` paired with `WeakRef` objects, which allows you to associate any type of input value with its respective (potentially large) computation result. See the WeakRefs and FinalizationRegistry example for more details.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`WeakMap`	36	12	6	23	8	36	6	24	8	3.0	37	8	1.0.0	1.0	0.12.0
`WeakMap()` constructor	36	12	6	23	8	36	6	24	8	3.0	37	8	1.0.0	1.0	0.12.0
`new WeakMap(iterable)`	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	4.0.0
`new WeakMap(null)`	36	12	37	23	8	36	37	24	8	3.0	37	8	1.0.0	1.0	0.12.0
`delete`	36	12	6[1]	23	8	36	6[2]	24	8	3.0	37	8	1.0.0	1.0	0.12.0
`get`	36	12	6[3]	23	8	36	6[4]	24	8	3.0	37	8	1.0.0	1.0	0.12.0
`getOrInsert`	145	145	144	129	26.2	145	144	96	26.2	**No**	145	26.2	1.2.20	2.6.7	26.0.0
`getOrInsertComputed`	145	145	144	129	26.2	145	144	96	26.2	**No**	145	26.2	1.2.20	2.6.7	26.0.0
`has`	36	12	6[1]	23	8	36	6[2]	24	8	3.0	37	8	1.0.0	1.0	0.12.0
`set`	36	12	6[1]	23	8	36	6[2]	24	8	3.0	37	8	1.0.0	1.0	0.12.0
Non-registered symbols as keys	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.1.0[5]

1. Before Firefox 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.
2. Before Firefox for Android 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.
3. Before Firefox 38, this method threw a `TypeError` when the key parameter was not an object. However, the ES2015 specification specifies to return `undefined` instead. Furthermore, `WeakMap.prototype.get` accepted an optional second argument as a fallback value, which is not part of the standard. Both non-standard behaviors are removed in version 38 and higher.
4. Before Firefox for Android 38, this method threw a `TypeError` when the key parameter was not an object. However, the ES2015 specification specifies to return `undefined` instead. Furthermore, `WeakMap.prototype.get` accepted an optional second argument as a fallback value, which is not part of the standard. Both non-standard behaviors are removed in version 38 and higher.
5. Also from version 18.20.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `WeakMap` in `core-js`
- Keyed collections
- `Map`
- `Set`
- `WeakSet`

JavaScript › Reference › Standard built-in objects › WeakMap

WeakMap() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap/WeakMap · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`WeakMap()`** constructor creates `WeakMap` objects.

Syntax

```js
new WeakMap()
new WeakMap(iterable)
```

> **Note:** `WeakMap()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`iterable`

An `Array` or other iterable object that produces a two-element array-like object whose first element is a value that will be used as a `WeakMap` key and whose second element is the value to associate with that key. Each key-value pair will be added to the new `WeakMap`. `null` is treated as `undefined`.

Examples

Using WeakMap

```js
const wm1 = new WeakMap();
const wm2 = new WeakMap();
const wm3 = new WeakMap();
const o1 = {};
const o2 = () => {};
const o3 = window;

wm1.set(o1, 37);
wm1.set(o2, "azerty");
wm2.set(o1, o2); // a value can be anything, including an object or a function
wm2.set(o3, undefined);
wm2.set(wm1, wm2); // keys and values can be any objects. Even WeakMaps!

wm1.get(o2); // "azerty"
wm2.get(o2); // undefined, because there is no key for o2 on wm2
wm2.get(o3); // undefined, because that is the set value

wm1.has(o2); // true
wm2.has(o2); // false
wm2.has(o3); // true (even if the value itself is 'undefined')

wm3.set(o1, 37);
wm3.get(o1); // 37

wm1.has(o1); // true
wm1.delete(o1);
wm1.has(o1); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`WeakMap()` constructor	36	12	6	23	8	36	6	24	8	3.0	37	8	1.0.0	1.0	0.12.0
`new WeakMap(iterable)`	38	12	36	25	9	38	36	25	9	3.0	38	9	1.0.0	1.0	4.0.0
`new WeakMap(null)`	36	12	37	23	8	36	37	24	8	3.0	37	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap.WeakMap`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `WeakMap` in `core-js`
- `WeakMap` in the JavaScript guide
- `Map`
- `Set`
- `WeakSet`

JavaScript › Reference › Standard built-in objects › WeakMap

WeakMap.prototype.delete()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap/delete · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since July 2015.

The **`delete()`** method of `WeakMap` instances removes the entry specified by the key from this `WeakMap`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: WeakMap.prototype.delete()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const weakmap = new WeakMap();
const object = {};

weakmap.set(object, 42);

console.log(weakmap.delete(object));
// Expected output: true

console.log(weakmap.has(object));
// Expected output: false
```

Syntax

```js
weakMapInstance.delete(key)
```

Parameters

key

The key of the entry to remove from the `WeakMap` object. Object keys are compared by reference, not by value.

Return value

`true` if an entry in the `WeakMap` object has been removed successfully. `false` if the key is not found in the `WeakMap`. Always returns `false` if `key` is not an object or a non-registered symbol.

Examples

Using delete()

```js
const wm = new WeakMap();
wm.set(window, "foo");

wm.delete(window); // Returns true. Successfully removed.

wm.has(window); // Returns false. The window object is no longer in the WeakMap.
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap.prototype.delete`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`delete`	36	12	6[1]	23	8	36	6[2]	24	8	3.0	37	8	1.0.0	1.0	0.12.0

1. Before Firefox 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.
2. Before Firefox for Android 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap.delete`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- WeakMap
- WeakMap.prototype.get()
- WeakMap.prototype.set()
- WeakMap.prototype.has()

JavaScript › Reference › Standard built-in objects › WeakMap

WeakMap.prototype.get()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap/get · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since July 2015.

The **get()** method of WeakMap instances returns the value corresponding to the key in this WeakMap, or undefined if there is none. Object values are returned as the same reference that was originally stored, not as a copy, so mutations to the returned object will be reflected anywhere that reference is held, including inside the WeakMap.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: WeakMap.prototype.get()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const weakmap = new WeakMap();
const object1 = {};
const object2 = {};

weakmap.set(object1, 42);

console.log(weakmap.get(object1));
// Expected output: 42

console.log(weakmap.get(object2));
// Expected output: undefined
```

Syntax

js

```
get(key)
```

Parameters

key

The key of the value to return from the WeakMap object. Object keys are compared by reference, not by value.

Return value

The value associated with the specified key in the WeakMap object. If the key can't be found, undefined is returned. Always returns undefined if key is not an object or a non-registered symbol.

Examples

Using get()

```js
const wm = new WeakMap();
wm.set(window, "foo");

wm.get(window); // Returns "foo".
wm.get("baz"); // Returns undefined.
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap.prototype.get

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
get	36	12	6[1]	23	8	36	6[2]	24	8	3.0	37	8	1.0.0	1.0	0.12.0

1. Before Firefox 38, this method threw a `TypeError` when the key parameter was not an object. However, the ES2015 specification specifies to return `undefined` instead. Furthermore, `WeakMap.prototype.get` accepted an optional second argument as a fallback value, which is not part of the standard. Both non-standard behaviors are removed in version 38 and higher.
2. Before Firefox for Android 38, this method threw a `TypeError` when the key parameter was not an object. However, the ES2015 specification specifies to return `undefined` instead. Furthermore, `WeakMap.prototype.get` accepted an optional second argument as a fallback value, which is not part of the standard. Both non-standard behaviors are removed in version 38 and higher.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap.get`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `WeakMap`
- `WeakMap.prototype.delete()`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.has()`

JavaScript › Reference › Standard built-in objects › WeakMap

WeakMap.prototype.getOrInsert()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap/getOrInsert · MDN page last modified 2026-10-26

Baseline 2026: Newly available — works across the latest browser versions since February 2026; may not work in older ones.

The **`getOrInsert()`** method of `WeakMap` instances returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

If the computation of the default value is expensive, consider using `WeakMap.prototype.getOrInsertComputed()` instead, which takes a callback to compute the default value only if it's actually needed.

Try it

▷ Interactive example on MDN: "JavaScript Demo: WeakMap.prototype.getOrInsert()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const map = new WeakMap([[window, "foo"]]);
console.log(map.getOrInsert(window, "default"));
// Expected output: "foo"

console.log(map.getOrInsert({}, "default"));
// Expected output: "default"
```

Syntax

js

```
getOrInsert(key, defaultValue)
```

Parameters

`key`

The key of the value to return from the `WeakMap` object. Must be either an object or a non-registered symbol. Object keys are compared by reference, not by value.

`defaultValue`

The value to insert and return if the key is not already present in the `WeakMap` object.

Return value

The value associated with the specified key in the `WeakMap` object. If the key can't be found, `defaultValue` is inserted and returned.

Exceptions

`TypeError`

Thrown if `key` is not an object or a non-registered symbol.

Examples

Using getOrInsert()

```js
const wm = new WeakMap();
const obj = {};

console.log(wm.get(obj)); // undefined
console.log(wm.getOrInsert(obj, "default")); // "default"
console.log(wm.get(obj)); // "default"
console.log(wm.getOrInsert(obj, "another default")); // "default"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap.prototype.getorinsert

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getOrInsert	145	145	144	129	26.2	145	144	96	26.2	**No**	145	26.2	1.2.20	2.6.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap.getOrInsert`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `WeakMap.prototype.getOrInsert` in `core-js`
- es-shims polyfill of `WeakMap.prototype.getOrInsert`
- `WeakMap`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.has()`
- `WeakMap.prototype.getOrInsertComputed()`

JavaScript › Reference › Standard built-in objects › WeakMap

WeakMap.prototype.getOrInsertComputed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap/getOrInsertComputed · MDN page last modified 2026-10-26

Baseline 2026: Newly available — works across the latest browser versions since February 2026; may not work in older ones.

The **`getOrInsertComputed()`** method of `WeakMap` instances returns the value corresponding to the specified key in this `WeakMap`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

Use this method instead of `WeakMap.prototype.getOrInsert()` when the default value is expensive to compute, and you want to avoid computing it unless it's actually needed.

Try it

▷ Interactive example on MDN: "JavaScript Demo: WeakMap.prototype.getOrInsertComputed()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const alan = { name: "Alan" };
const map = new WeakMap([[alan, 35]]);
const defaultCreator = (obj) => `${obj.name}'s age is unknown`;

console.log(map.getOrInsertComputed(alan, defaultCreator));
// Expected output: 35

const brett = { name: "Brett" };
console.log(map.getOrInsertComputed(brett, defaultCreator));
// Expected output: "Brett's age is unknown"
```

Syntax

js

```
getOrInsertComputed(key, callback)
```

Parameters

`key`

The key of the element to return from the `Map` object. Must be either an object or a non-registered symbol. Object keys are compared by reference, not by value.

`callback`

A function that returns the value to insert and return if the key is not already present in the `Map` object. The function is called with the following argument:

`key`

The same key that was passed to `getOrInsertComputed()`.

Return value

The value associated with the specified key in the `WeakMap` object. If the key can't be found, the result of `callback(key)` is inserted and returned.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `key` is not an object or a non-registered symbol.
- `callback` isn't callable.

Examples

Caching

Caching involves building a mapping from the parameter of an expensive function to its return value, so that future calls with the same parameter can return the cached value instead of recomputing it. When no cached value exists, it needs to be computed and inserted into the cache.

We use a `WeakMap` here instead of a `Map`, so that the cached argument values do not prevent them from being garbage-collected if they are no longer needed elsewhere in the program. If your cached function takes non-object arguments, you can use a `Map` instead.

```js
// Any expensive function you want to cache
function computeExpensiveValue(requestOptions) {
  // Imagine expensive computation, like fetching data or complex calculations
  console.log(`Fetching from ${requestOptions.url}`);
  return new Response("Fake response");
}

// A higher-order function that adds caching to any function
function withCache(fn) {
  const cache = new WeakMap();
  return (param) => cache.getOrInsertComputed(param, fn);
}

const computeWithCache = withCache(computeExpensiveValue);
const options = { url: "https://example.com/a", method: "GET" };
const value1 = computeWithCache(options); // Logs "Fetching from https://example.com/a"
const value2 = computeWithCache(options); // No log
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap.prototype.getorinsertcomputed`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getOrInsertComputed`	145	145	144	129	26.2	145	144	96	26.2	**No**	145	26.2	1.2.20	2.6.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap.getOrInsertComputed`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `WeakMap.prototype.getOrInsertComputed` in `core-js`
- es-shims polyfill of `WeakMap.prototype.getOrInsertComputed`
- `WeakMap`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.has()`
- `WeakMap.prototype.getOrInsert()`

JavaScript › Reference › Standard built-in objects › WeakMap

WeakMap.prototype.has()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap/has · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since July 2015.

The **`has()`** method of `WeakMap` instances returns a boolean indicating whether an entry with the specified key exists in this `WeakMap` or not.

Try it

▷ Interactive example on MDN: "JavaScript Demo: WeakMap.prototype.has()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const weakmap = new WeakMap();
const object1 = {};
const object2 = {};

weakmap.set(object1, "foo");

console.log(weakmap.has(object1));
// Expected output: true

console.log(weakmap.has(object2));
// Expected output: false
```

Syntax

js

```
has(key)
```

Parameters

key

The key of the entry to test for presence in the `WeakMap` object. Object keys are compared by reference, not by value.

Return value

Returns `true` if an entry with the specified key exists in the `WeakMap` object; otherwise `false`. Always returns `false` if `key` is not an object or a non-registered symbol.

Examples

Using has()

```js
const wm = new WeakMap();
wm.set(window, "foo");

wm.has(window); // returns true
wm.has("baz"); // returns false
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap.prototype.has`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
has	36	12	6[1]	23	8	36	6[2]	24	8	3.0	37	8	1.0.0	1.0	0.12.0

1. Before Firefox 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.
2. Before Firefox for Android 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap.has`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `WeakMap`
- `WeakMap.prototype.delete()`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.set()`

JavaScript › Reference › Standard built-in objects › WeakMap

WeakMap.prototype.set()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap/set · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since July 2015.

The **`set()`** method of WeakMap instances adds a new entry with a specified key and value to this `WeakMap`, or updates an existing entry if the key already exists.

Try it

▷ Interactive example on MDN: "JavaScript Demo: WeakMap.prototype.set()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const weakmap = new WeakMap();
const object1 = {};
const object2 = {};

weakmap.set(object1, "foo");
weakmap.set(object2, "bar");

console.log(weakmap.get(object1));
// Expected output: "foo"

console.log(weakmap.get(object2));
// Expected output: "bar"
```

Syntax

js

```
set(key, value)
```

Parameters

`key`

The key of the entry to add to or modify within the `WeakMap` object. Must be either an object or a non-registered symbol. Object keys are compared by reference, not by value.

`value`

The value of the entry to add to or modify within the `WeakMap` object. Can be any value.

Return value

The `WeakMap` object.

Exceptions

`TypeError`

Thrown if `key` is not an object or a non-registered symbol.

Examples

Using set()

```js
const wm = new WeakMap();
const obj = {};

// Add new elements to the WeakMap
wm.set(obj, "foo").set(window, "bar"); // chainable

// Update an element in the WeakMap
wm.set(obj, "baz");

// Using a non-registered symbol as key
const sym = Symbol("foo");
wm.set(sym, "baz");
wm.set(Symbol.iterator, "qux");
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakmap.prototype.set

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
set	36	12	6[1]	23	8	36	6[2]	24	8	3.0	37	8	1.0.0	1.0	0.12.0

1. Before Firefox 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.
2. Before Firefox for Android 38, this method threw a `TypeError` when the key parameter was not an object. This has been fixed in version 38 and later to return `false` as per the ES2015 standard.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakMap.set`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `WeakMap`
- `WeakMap.prototype.delete()`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.has()`

JavaScript › Reference › Standard built-in objects

WeakRef

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakRef · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

A **`WeakRef`** object lets you hold a weak reference to another object, without preventing that object from getting garbage-collected.

Description

A `WeakRef` object contains a weak reference to an object, which is called its *target* or *referent*. A *weak reference* to an object is a reference that does not prevent the object from being reclaimed by the garbage collector. In contrast, a normal (or *strong*) reference keeps an object in memory. When an object no longer has any strong references to it, the JavaScript engine's garbage collector may destroy the object and reclaim its memory. If that happens, you can't get the object from a weak reference anymore.

Because non-registered symbols are also garbage collectable, they can also be used as the target of a `WeakRef` object. However, the use case of this is limited.

Avoid where possible

Correct use of `WeakRef` takes careful thought, and it's best avoided if possible. It's also important to avoid relying on any specific behaviors not guaranteed by the specification. When, how, and whether garbage collection occurs is down to the implementation of any given JavaScript engine. Any behavior you observe in one engine may be different in another engine, in another version of the same engine, or even in a slightly different situation with the same version of the same engine. Garbage collection is a hard problem that JavaScript engine implementers are constantly refining and improving their solutions to.

Here are some specific points included by the authors in the proposal that introduced `WeakRef`:

> Garbage collectors are complicated. If an application or library depends on GC cleaning up a WeakRef or calling a finalizer [cleanup callback] in a timely, predictable manner, it's likely to be disappointed: the cleanup may happen much later than expected, or not at all. Sources of variability include:
>
> - One object might be garbage-collected much sooner than another object, even if they become unreachable at the same time, e.g., due to generational collection.
> - Garbage collection work can be split up over time using incremental and concurrent techniques.
> - Various runtime heuristics can be used to balance memory usage, responsiveness.
> - The JavaScript engine may hold references to things which look like they are unreachable (e.g., in closures, or inline caches).
> - Different JavaScript engines may do these things differently, or the same engine may change its algorithms across versions.
> - Complex factors may lead to objects being held alive for unexpected amounts of time, such as use with certain APIs.

Notes on WeakRefs

- If your code has just created a `WeakRef` for a target object, or has gotten a target object from a `WeakRef`'s `deref` method, that target object will not be reclaimed until the end of the current

JavaScript job (including any promise reaction jobs that run at the end of a script job). That is, you can only "see" an object get reclaimed between turns of the event loop. This is primarily to avoid making the behavior of any given JavaScript engine's garbage collector apparent in code — because if it were, people would write code relying on that behavior, which would break when the garbage collector's behavior changed. (Garbage collection is a hard problem; JavaScript engine implementers are constantly refining and improving how it works.)

- If multiple `WeakRef`s have the same target, they're consistent with one another. The result of calling `deref` on one of them will match the result of calling `deref` on another of them (in the same job), you won't get the target object from one of them but `undefined` from another.
- If the target of a `WeakRef` is also in a `FinalizationRegistry`, the `WeakRef`'s target is cleared at the same time or before any cleanup callback associated with the registry is called; if your cleanup callback calls `deref` on a `WeakRef` for the object, it will receive `undefined`.
- You cannot change the target of a `WeakRef`, it will always only ever be the original target object or `undefined` when that target has been reclaimed.
- A `WeakRef` might never return `undefined` from `deref`, even if nothing strongly holds the target, because the garbage collector may never decide to reclaim the object.

Constructor

`WeakRef()`

Creates a new `WeakRef` object.

Instance properties

These properties are defined on `WeakRef.prototype` and shared by all `WeakRef` instances.

`WeakRef.prototype.constructor` Optional

The constructor function that created the instance object. For `WeakRef` instances, the initial value is the `WeakRef` constructor.

> **Note:** This property is marked as "normative optional" in the specification, which means a conforming implementation may not expose the `constructor` property. This prevents arbitrary code from obtaining the `WeakRef` constructor and being able to observe garbage collection. However, all major engines do expose it by default.

`WeakRef.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"WeakRef"`. This property is used in `Object.prototype.toString()`.

Instance methods

`WeakRef.prototype.deref()`

Returns the `WeakRef` object's target object, or `undefined` if the target object has been reclaimed.

Examples

Using a WeakRef object

This example starts a counter shown in a DOM element, stopping when the element doesn't exist anymore:

js

```
class Counter {
  constructor(element) {
    // Remember a weak reference to the DOM element
    this.ref = new WeakRef(element);
    this.start();
  }

  start() {
    if (this.timer) {
      return;
    }

    this.count = 0;

    const tick = () => {
      // Get the element from the weak reference, if it still exists
      const element = this.ref.deref();
      if (element) {
        element.textContent = ++this.count;
      } else {
        // The element doesn't exist anymore
        console.log("The element is gone.");
        this.stop();
        this.ref = null;
      }
    };

    tick();
    this.timer = setInterval(tick, 1000);
  }

  stop() {
    if (this.timer) {
      clearInterval(this.timer);
      this.timer = 0;
    }
  }
}

const counter = new Counter(document.getElementById("counter"));
setTimeout(() => {
  document.getElementById("counter").remove();
}, 5000);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/managing-memory.html#sec-weak-ref-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`WeakRef`	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
`WeakRef()` constructor	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
Non-registered symbol as target	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.0.0 [1]
`deref`	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0

1. Also from version 18.20.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakRef`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `FinalizationRegistry`
- `WeakSet`
- `WeakMap`

JavaScript › Reference › Standard built-in objects › WeakRef

WeakRef() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakRef/WeakRef · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`WeakRef()`** constructor creates `WeakRef` objects.

Syntax

```js
new WeakRef(target)
```

> **Note:** `WeakRef()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`target`

The target value the WeakRef should refer to (also called the *referent*). Must be an object or a non-registered symbol.

Return value

A new `WeakRef` object referring to the given target value.

Exceptions

`TypeError`

Thrown if `target` is not an object or a non-registered symbol.

Examples

Creating a new WeakRef object

See the main WeakRef page for a complete example.

```js
class Counter {
  constructor(element) {
    // Remember a weak reference to a DOM element
    this.ref = new WeakRef(element);
    this.start();
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/managing-memory.html#sec-weak-ref-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
WeakRef() constructor	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
Non-registered symbol as target	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.0.0 [1]

1. Also from version 18.20.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakRef.WeakRef`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- WeakRef

JavaScript › Reference › Standard built-in objects › WeakRef

WeakRef.prototype.deref()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakRef/deref · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`deref()`** method of WeakRef instances returns this `WeakRef`'s target value, or `undefined` if the target value has been garbage-collected.

Syntax

```js
deref()
```

Parameters

None.

Return value

The target value of the WeakRef, which is either an object or a non-registered symbol. Returns `undefined` if the value has been garbage-collected.

Description

See the Notes on WeakRefs section of the `WeakRef` page for some important notes.

Examples

Using deref()

See the Examples section of the `WeakRef` page for the complete example.

```js
const tick = () => {
  // Get the element from the weak reference, if it still exists
  const element = this.ref.deref();
  if (element) {
    element.textContent = ++this.count;
  } else {
    // The element doesn't exist anymore
    console.log("The element is gone.");
    this.stop();
    this.ref = null;
  }
};
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/managing-memory.html#sec-weak-ref.prototype.deref`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`deref`	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakRef.deref`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `WeakRef`

JavaScript › Reference › Standard built-in objects

WeakSet

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakSet · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

A **`WeakSet`** is a collection of garbage-collectable values, including objects and non-registered symbols. A value in the `WeakSet` may only occur once. It is unique in the `WeakSet`'s collection.

Description

Values of WeakSets must be garbage-collectable. Most primitive data types can be arbitrarily created and don't have a lifetime, so they cannot be stored. Objects and non-registered symbols can be stored because they are garbage-collectable.

The main differences to the `Set` object are:

- `WeakSet`s are collections of **objects and symbols only**. They cannot contain arbitrary values of any type, as `Set`s can.
- The `WeakSet` is *weak*, meaning references to objects in a `WeakSet` are held *weakly*. If no other references to a value stored in the `WeakSet` exist, those values can be garbage collected.

> **Note:** This also means that there is no list of current values stored in the collection. `WeakSet`s are not enumerable.

Key equality

Like regular `Set`, value equality is based on the SameValueZero algorithm, which is the same as the `===` operator because `WeakSet` can only hold object and symbol values. This means that for object values, equality is based on object identity. They are compared by reference, not by value.

Constructor

`WeakSet()`

Creates a new `WeakSet` object.

Instance properties

These properties are defined on `WeakSet.prototype` and shared by all `WeakSet` instances.

`WeakSet.prototype.constructor`

The constructor function that created the instance object. For `WeakSet` instances, the initial value is the `WeakSet` constructor.

`WeakSet.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"WeakSet"`. This property is used in `Object.prototype.toString()`.

Instance methods

`WeakSet.prototype.add()`

Inserts the specified value into this set, if it is not already present.

`WeakSet.prototype.delete()`

Removes the specified value from this set, if it is in the set.

`WeakSet.prototype.has()`

Returns a boolean indicating whether the specified value exists in this `WeakSet` or not.

Examples

Using the WeakSet object

```js
const ws = new WeakSet();
const foo = {};
const bar = {};

ws.add(foo);
ws.add(bar);

ws.has(foo); // true
ws.has(bar); // true

ws.delete(foo); // removes foo from the set
ws.has(foo); // false, foo has been removed
ws.has(bar); // true, bar is retained
```

Note that `foo !== bar`. While they are similar objects, *they are not **the same object***. And so they are both added to the set.

Detecting circular references

Functions that call themselves recursively need a way of guarding against circular data structures by tracking which objects have already been processed.

`WeakSet`s are ideal for this purpose:

```js
// Execute a callback on everything stored inside an object
function execRecursively(fn, subject, _refs = new WeakSet()) {
  // Avoid infinite recursion
  if (_refs.has(subject)) {
    return;
  }

  fn(subject);
  if (typeof subject === "object" && subject) {
    _refs.add(subject);
    for (const key in subject) {
      execRecursively(fn, subject[key], _refs);
    }
    _refs.delete(subject);
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
const foo = {
  foo: "Foo",
  bar: {
    bar: "Bar",
  },
};

foo.bar.baz = foo; // Circular reference!
execRecursively((obj) => console.log(obj), foo);
```

Here, a `WeakSet` is created on the first run, and passed along with every subsequent function call (using the internal `_refs` parameter).

The number of objects or their traversal order is immaterial, so a `WeakSet` is more suitable (and performant) than a `Set` for tracking object references, especially if a very large number of objects is involved.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakset-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`WeakSet`	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0
`WeakSet()` constructor	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0
`new WeakSet(iterable)`	38	12	34	25	9	38	34	25	9	3.0	38	9	1.0.0	1.0	4.0.0
`new WeakSet(null)`	36	12	37	23	9	36	37	24	9	3.0	37	9	1.0.0	1.0	0.12.0
`add`	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0
`delete`	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0
`has`	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0
Non-registered symbols as keys	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.0.0 [1]

1. Also from version 18.20.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakSet`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `WeakSet` in `core-js`
- `Map`
- `Set`
- `WeakMap`

JavaScript › Reference › Standard built-in objects › WeakSet

WeakSet() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakSet/WeakSet · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`WeakSet()`** constructor creates `WeakSet` objects.

Syntax

```js
new WeakSet()
new WeakSet(iterable)
```

> **Note:** `WeakSet()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`iterable` Optional

If an iterable object is passed, all of its elements will be added to the new `WeakSet`. `null` is treated as `undefined`.

Examples

Using the WeakSet object

```js
const ws = new WeakSet();
const foo = {};
const bar = {};

ws.add(foo);
ws.add(bar);

ws.has(foo); // true
ws.has(bar); // true

ws.delete(foo); // removes foo from the set
ws.has(foo); // false, foo has been removed
ws.has(bar); // true, bar is retained
```

Note that `foo !== bar`. While they are similar objects, *they are not **the same object***. And so they are both added to the set.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakset-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`WeakSet()` constructor	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0
`new WeakSet(iterable)`	38	12	34	25	9	38	34	25	9	3.0	38	9	1.0.0	1.0	4.0.0
`new WeakSet(null)`	36	12	37	23	9	36	37	24	9	3.0	37	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakSet.WeakSet`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `WeakSet` in `core-js`
- `WeakSet`

JavaScript › Reference › Standard built-in objects › WeakSet

WeakSet.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakSet/add · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since September 2015.

The **`add()`** method of `WeakSet` instances inserts the specified value into this set, if it is not already present.

Try it

▷ Interactive example on MDN: "JavaScript Demo: WeakSet.prototype.add()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const weakset = new WeakSet();
const object = {};

weakset.add(object);
console.log(weakset.has(object));
// Expected output: true

try {
  weakset.add(1);
} catch (error) {
  console.log(error);
  // Expected output (Chrome): TypeError: Invalid value used in weak set
  // Expected output (Firefox): TypeError: WeakSet value must be an object, got 1
  // Expected output (Safari): TypeError: Attempted to add a non-object key to a WeakSet
}
```

Syntax

```
add(value)
```

Parameters

`value`

The value to add to the `WeakSet` object. Must be either an object or a non-registered symbol. Objects are compared by reference, not by value.

Return value

The `WeakSet` object.

Exceptions

`TypeError`

Thrown if `value` is not an object or a non-registered symbol.

Examples

Using add()

```
const ws = new WeakSet();

ws.add(window); // add the window object to the WeakSet

ws.has(window); // true

// WeakSet only takes objects as arguments
ws.add(1);
// results in "TypeError: Invalid value used in weak set" in Chrome
// and "TypeError: 1 is not a non-null object" in Firefox
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakset.prototype.add`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakSet.add`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `WeakSet`
- `WeakSet.prototype.delete()`
- `WeakSet.prototype.has()`

JavaScript › Reference › Standard built-in objects › WeakSet

WeakSet.prototype.delete()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakSet/delete · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since September 2015.

The **`delete()`** method of `WeakSet` instances removes the specified value from this set, if it is in the set.

Try it

▷ Interactive example on MDN: "JavaScript Demo: WeakSet.prototype.delete()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const weakset = new WeakSet();
const object = {};

weakset.add(object);

console.log(weakset.has(object));
// Expected output: true

weakset.delete(object);

console.log(weakset.has(object));
// Expected output: false
```

Syntax

js

```
weakSetInstance.delete(value)
```

Parameters

`value`

The value to remove from the `WeakSet` object. Objects are compared by reference, not by value.

Return value

`true` if a value in the `WeakSet` object has been removed successfully. `false` if the value is not found in the `WeakSet`. Always returns `false` if `value` is not an object or a non-registered symbol.

Examples

Using delete()

```js
const ws = new WeakSet();
const obj = {};

ws.add(window);

ws.delete(obj); // Returns false. No obj found to be deleted.
ws.delete(window); // Returns true. Successfully removed.

ws.has(window); // Returns false. The window is no longer present in the WeakSet.
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakset.prototype.delete

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
delete	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakSet.delete`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `WeakSet`
- `WeakSet.prototype.add()`
- `WeakSet.prototype.has()`

JavaScript › Reference › Standard built-in objects › WeakSet

WeakSet.prototype.has()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakSet/has · MDN page last modified 2025-09-22

Baseline: Widely available — available across major browsers since September 2015.

The **`has()`** method of `WeakSet` instances returns a boolean indicating whether the specified value exists in this `WeakSet` or not.

Try it

▷ Interactive example on MDN: "JavaScript Demo: WeakSet.prototype.has()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const weakset = new WeakSet();
const object1 = {};
const object2 = {};

weakset.add(object1);

console.log(weakset.has(object1));
// Expected output: true

console.log(weakset.has(object2));
// Expected output: false
```

Syntax

js

```
has(value)
```

Parameters

`value`

The value to test for presence in the `WeakSet` object. Objects are compared by reference, not by value.

Return value

Returns `true` if the specified value exists in the `WeakSet` object; otherwise `false`. Always returns `false` if `value` is not an object or a non-registered symbol.

Examples

Using has()

js

```
const ws = new WeakSet();
const obj = {};
ws.add(window);

ws.has(window); // returns true
ws.has(obj); // returns false

// Storing a non-registered symbol
const sym = Symbol("foo");
ws.add(sym);
ws.add(Symbol.iterator);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/keyed-collections.html#sec-weakset.prototype.has`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
has	36	12	34	23	9	36	34	24	9	3.0	37	9	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.WeakSet.has`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `WeakSet`
- `WeakSet.prototype.add()`
- `WeakSet.prototype.delete()`